**Department of Computer Science**

**Software Engineering**

**Final Project (30%)**

**Phase 2: Software Design Specification Document (10%)**

# Rihla

Desert and City Experience Booking Platform for Dubai

## Software Design Specification (SDS)


### Team members and parts

Salem Al Tamimi: Sections 1 and 2

Hamzah and Hamad: Section 3

Mir Umer: Section 4

Omar Arif: Section 5

Oumar Ibrahim: Section 6


*Built with Python 3.12: a Tkinter desktop client, a Flask REST server and an SQLite database*

# Table of Contents

1.0 Introduction ..... 4
1.1 Goals and objectives ..... 4
1.2 Statement of scope ..... 5
1.3 Software context ..... 6
1.4 Major constraints ..... 7
2.0 Data design ..... 8
2.1 Internal software data structure ..... 8
2.2 Global data structure ..... 9
2.3 Temporary data structure ..... 9
2.4 Database description ..... 10
3.0 Architectural and component-level design ..... 16
3.1 System Structure ..... 17
3.2 Description for Components ..... 21
3.2 Description for Component “Accounts and Sign-in” ..... 22
3.2 Description for Component “Events and Venues” ..... 25
3.2 Description for Component “Booking and Payment” ..... 27
3.2 Description for Component “Cancellations and Refunds” ..... 32
3.2 Description for Component “Venue Operations” ..... 37
3.2 Description for Component “Fleet and Dispatch” ..... 40
3.2 Description for Component “Administration, Reports and Reviews” ..... 42
3.3 Dynamic Behavior for Components ..... 46
4.0 User interface design ..... 48
4.1 Description of the user interface ..... 48
4.2 Interface design rules ..... 59
4.3 Components available ..... 60

4.4 UIDS description ..... 60
5.0 Restrictions, limitations, and constraints ..... 61
6.0 Testing Issues ..... 63
6.1 Classes of tests ..... 63
6.2 Expected software response ..... 64
6.3 Performance bounds ..... 65
6.4 Identification of critical components ..... 65
7.0 Work distribution ..... 66

Section 1

# 1.0 Introduction

Rihla is a booking platform for experiences in Dubai, such as evening desert safaris, dinners with a view, yacht cruises and overnight camps. In Phase 1 we wrote down what the system should do. This document explains how we actually built it: how the data is stored, how the program is split into parts, what the screens look like and how we test it.

One thing changed a lot since Phase 1. Our first plan was to write a custom module for Odoo on top of PostgreSQL. When we started coding, we decided that a full business framework was more than a team of our size could learn and finish in one semester, so we moved to plain Python. Rihla is now a Tkinter desktop app that talks to a small Flask server, and the server keeps everything in one SQLite file. Everything in this document describes that final version.

## 1.1 Goals and objectives

Small tour companies in Dubai still take many bookings over the phone and on WhatsApp. When a tourist, a hotel desk and a travel agency all want the last seats of the same safari, somebody usually gets a seat that does not exist. The main goal of Rihla is to stop this by keeping every booking in one place and checking the seats on the server, at the moment of booking.

The second goal is to give each person in the process a screen made for their job. A tourist does not need to see the vehicle fleet, and the driver team does not need the payment window. We ended up with six roles: Customer, Vendor, Staff, Partner (travel agency), Dispatcher and Admin.

Our objectives are:

- **No double booking:** seats are counted again inside a database transaction when the booking is saved.
- **Honest prices:** the 5% UAE VAT is always shown as its own line, and the user can see prices in AED, USD, EUR or GBP.

- **Simple tickets:** every booking gets a 16-digit code that venue staff can check in once.
- **A clear refund policy:** the same cancellation rules for everybody, with a record of who cancelled and why.
- **Safety:** an SSN for every account that is not a customer, a Report button for everyone, and bans handled by the admin.

## 1.2 Statement of scope

Rihla follows a booking from start to end. A vendor publishes an event and the dispatcher gives it vehicles. A customer or a partner books it and pays. On the day, staff at the venue check the ticket in, and afterwards the customer can leave a review. Cancellations, refunds, support questions and reports also live inside the system.

The main inputs are account details, event details (title, category, price in AED, venue, date or weekly days, time, capacity and a picture), booking requests, payment details, ticket codes, vehicle data, support messages and reports. The main outputs are the event list with live seats, the booking confirmation with its ticket code, a VAT breakdown, a PDF receipt, check-in results, the day's schedule for a venue and the refund amount when a booking is cancelled.

We ranked the functions from our point of view as developers. Essential means it is finished and tested. Desirable means we wanted it and got most of it done. Future means it is not in the code yet.

| Function | Priority | Where it stands |
|---|---|---|
| Sign up, log in and password reset for six roles | Essential | Done. Passwords are salted PBKDF2 hashes. |
| 16-digit SSN for non-customer accounts | Essential | Done. Only a hash and the last 4 digits are kept. |
| Events, venues and categories for vendors | Essential | Done, with pictures and weekly repeats. |
| Explore, search and filter tours | Essential | Done (category, maximum price, search box). |
| Seat check for each date | Essential | Done. Uses vehicle seats or the event capacity. |
| Booking with 5% VAT and a currency choice | Essential | Done. Totals are saved in AED with VAT. |

| Function | Priority | Where it stands |
|---|---|---|
| Payment by Visa, Mastercard or bank transfer | Essential | Demo only. Checked against fixed details. |
| Booking Details window with PDF download | Essential | Done. The PDF is written by our own code. |
| Ticket check-in by venue staff | Essential | Done. Staff only see their own venue. |
| Cancellation and refund rules | Essential | Done. 100%, 50% or not allowed, with a record. |
| Partner shopping list paid in one go | Essential | Done. All items or none are booked. |
| Vehicle register and event dispatch | Essential | Done. Vehicle seats become the capacity. |
| Admin: accounts, bans, reports, all bookings | Essential | Done. |
| Reviews and average ratings | Desirable | Done, 0 to 5 stars in half steps. |
| Support tickets answered at the venue | Desirable | Done. Staff can mark them resolved. |
| Google Maps link for each venue | Desirable | Done. |
| Real email and SMS codes | Desirable | Not done. The code is always 000000. |
| Real payment gateway and real refunds | Future | Not done. |
| QR tickets, driver app, live vehicle map | Future | Not done. |
| Arabic and other languages | Future | Not done. English only. |

*Table 1: Ranking of the major functions*

## 1.3 Software context

Tourism is one of the largest industries in Dubai, and most visitors book at least one desert or sea trip. The companies that run these trips are often small and sell through many channels at once: their own customers, hotel desks and travel agencies. Rihla puts all of these on one system. Customers book for themselves. Partners book groups for their clients and are allowed more than 10 guests per booking. Vendors see every booking for their events, dispatchers decide how many seats an event has by giving it vehicles, and staff at the venue see who is coming today.

On the technical side, Rihla is a client and server program (Figure 1). The client in the src folder is a Tkinter app that the user starts with start_client.py. It never opens the database itself. Every request goes through api_client.py as HTTP with JSON to the Flask server in the server folder, which listens on 127.0.0.1 port 5000. Because only the server touches users.db, all the rules about seats, VAT, refunds, bans and venues are written once, in one place.

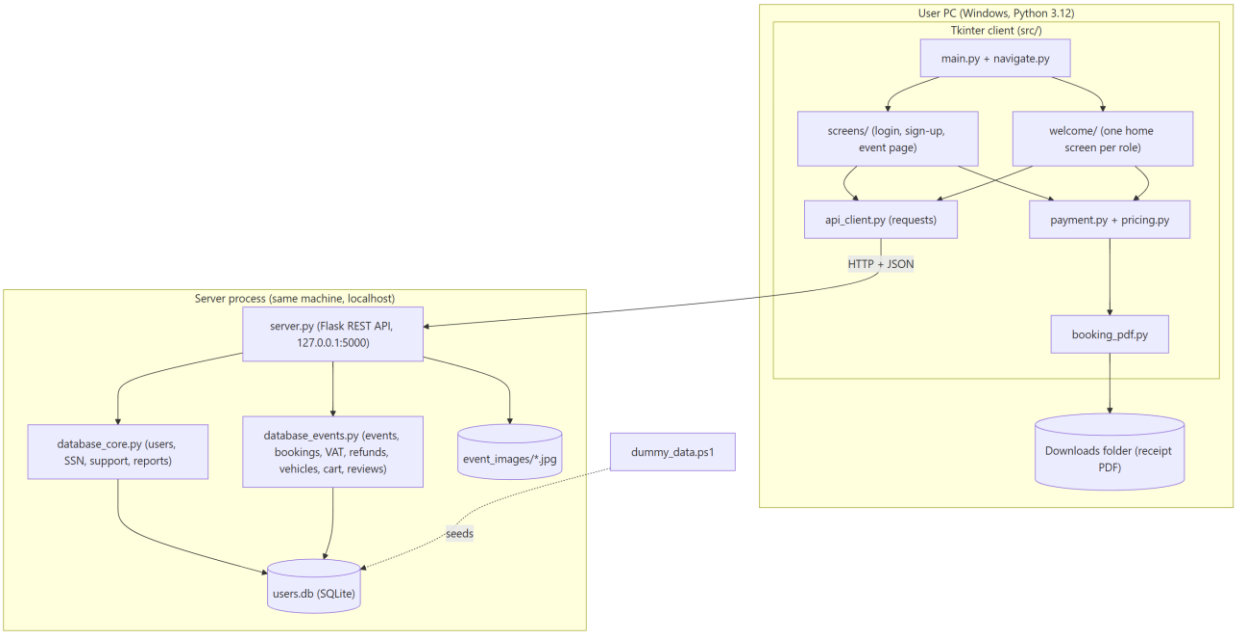



*Figure 1: Component and deployment view of Rihla*

## 1.4 Major constraints

A few things shaped the design more than others.

- **Time and team size:** we had one semester, so we chose tools the whole team already knew (Python, Tkinter, Flask and SQLite) instead of learning a big framework.
- **UAE VAT:** VAT is 5% and has to appear as a separate line. All money is kept in AED. Other currencies are only for showing prices.
- **No real money:** a course project cannot process real cards, so the payment step is a demo that compares the form with fixed demo details.
- **Privacy:** an SSN is sensitive, so the server never stores the number itself.
- **Windows:** the team works on Windows 10 and 11, so that is what we tested on. A few details, like finding the Downloads folder, use the Windows registry.
- **Easy to demo:** the server creates six demo accounts when it starts, and dummy_data.ps1 fills the database with events, bookings and reviews whose dates are relative to today.

Section 2

# 2.0 Data design

This section describes the data inside Rihla: what moves between the parts of the program, what is shared by all of them, which files are only kept for a while, and how the database is built.

## 2.1 Internal software data structure

Inside Rihla, data travels as Python dictionaries. Between the client and the server the same dictionaries are sent as JSON. Every answer from the server has a success flag, and when something goes wrong it also has a message written for the user, which the client shows as it is. This kept our error handling simple, because the screens never need to guess what went wrong.

The most important structure is the event. It holds the title, category, price, description, venue (with its name and map link), the date for a one-time event or the weekdays and time for a repeating one, and the capacity. Almost every screen starts from a list of events.

Close to it is the capacity info, which the event page asks for each time the user picks a date. It says how many seats the event has in total, where that number comes from (the assigned vehicles or the event itself), how many are already booked on that date and how many remain. The capacity bar on the screen is drawn straight from these numbers.

The booking is the structure that most roles share. Besides the event, date, guests, total price and ticket code, it carries a status and the cancellation details. The status is worked out when the booking is read, not stored as such. The database only keeps 'Confirmed' or 'Cancelled' for the booking and 'Valid', 'Used' or 'Cancelled' for its ticket. The function _booking_dict combines the two with today's date and shows the user 'Confirmed', 'Checked in', 'Expired' or 'Cancelled'. It also adds can_cancel and refund_if_cancelled, so the screen can show the refund before the user decides.

Smaller structures connect single steps. The payment result (txn_id, method, currency) goes from the payment window to the Booking Details window. The receipt is a list of lines, each with its text, font size and bold flag, and the same list is used for the window and for the PDF. A refund is returned as percent, base, VAT and total. A cart item, a vehicle assignment and a user row for the admin table follow the same idea.

## 2.2 Global data structure

The global store is one SQLite database, server/users.db. No screen opens it directly. The client asks the server, and the server uses database_core.py (accounts, SSN, staff venues, support tickets and reports) or database_events.py (everything about events and bookings). Event pictures are kept next to it as files in server/event_images, named by the event id.

A few modules hold values that large parts of the program depend on:

- **config.py:** the six roles, the demo accounts, the roles allowed at sign-up (all except Admin) and the minimum password length of 8. There is one copy for the client and one for the server.
- **pricing.py:** VAT_RATE = 0.05 and the fixed display rates for 1 AED (USD 0.2723, EUR 0.2510, GBP 0.2150).
- **database_events.py:** the refund rules (48 and 24 hours) and the 300 character limit for a cancel reason.
- **database_core.py:** default capacity 20, maximum capacity 5000, 10 guests per customer booking, 3 open support tickets per customer and SSN length 16.
- **theme.py:** colours and the Segoe UI font, so every screen matches.

## 2.3 Temporary data structure

Some data only lives for a short time:

- **Partner cart:** the partner_cart table holds the events a partner has picked but not paid for. The rows are deleted after a successful checkout.

- **Picture cache:** the client keeps downloaded event pictures in memory and drops one when the vendor changes it, so lists do not download the same picture again.
- **Uploaded picture:** before upload the client shrinks the picture to at most 1400 pixels and sends it as base64 inside the JSON. The server checks that it is a JPEG under 4 MB before writing the file.
- **Receipt PDF:** saved in the user's Downloads folder as Rihla_Booking_<transaction id>.pdf. If the file exists, a number is added to the name, so old receipts are never overwritten.
- **Test databases:** the unit tests point the database path to a temporary file, so they never touch the real data.
- **Seed record:** dummy_data.ps1 writes down every row it adds in a demo_seed table, so the next run can remove exactly those rows and nothing else.

## 2.4 Database description

The database is created by init_db in database_core.py. It runs every time the server starts. It creates the tables that are missing, adds new columns to databases made by older versions of our code (for example the cancellation columns), and fills in the default data: five vehicle types, two venues, three categories, two demo events, seven vehicles, a demo ticket and the six demo accounts. The reviews table is the exception. It is created the first time someone reads or writes a review.

Figure 2 shows the tables and how they relate. We should be honest about one design choice here. The users table has no number id, so most tables point to a user by email address (for example customer_email or vendor_email). This was quick to build, but it means an email cannot be changed later without touching many tables. Section 5 comes back to this.

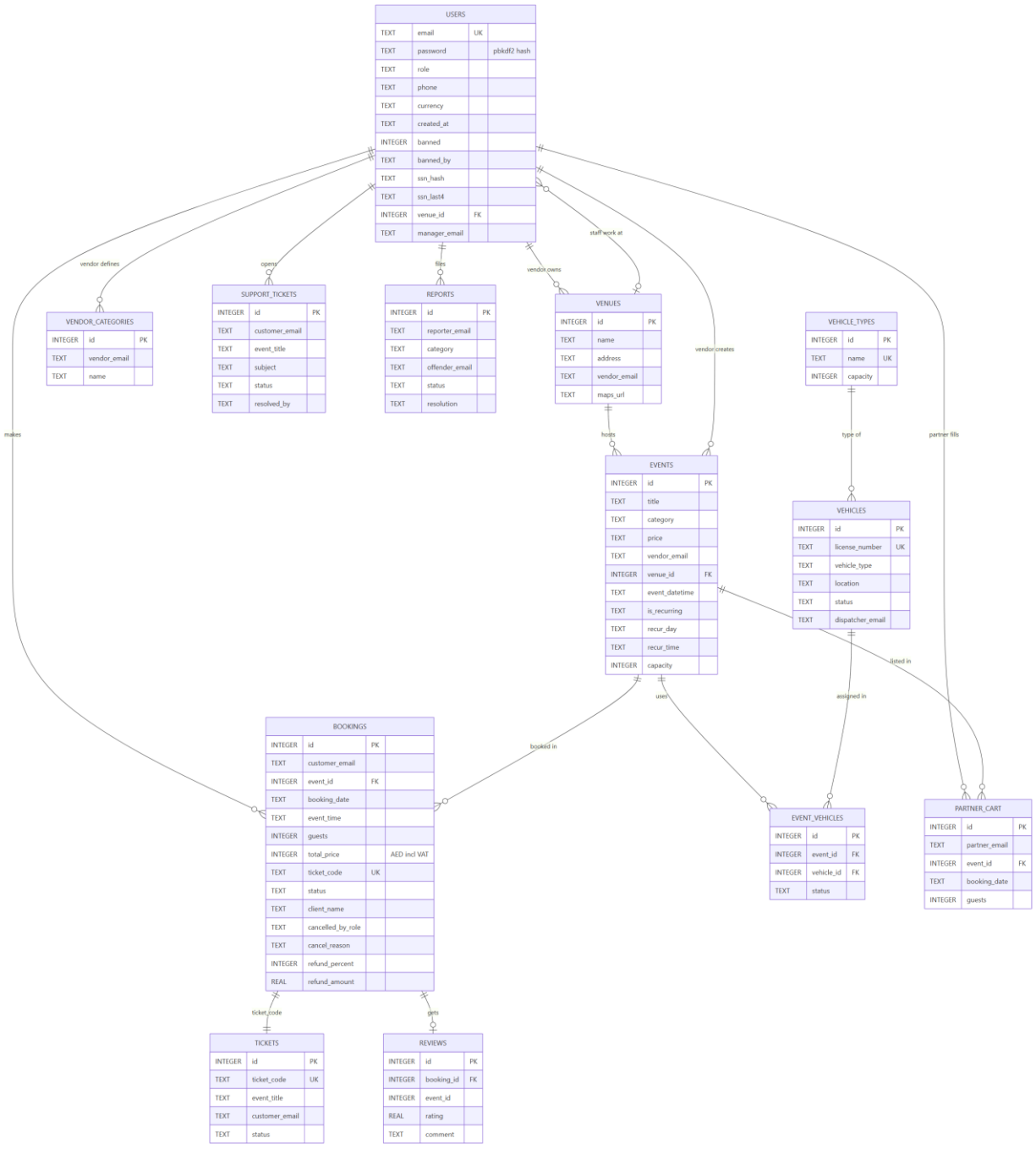



*Figure 2: Entity relationship diagram of server/users.db*

The tables below give enough detail to create the database again.

## Table Name: users

| Attribute | Data Type | Constraint |
|---|---|---|
| email | TEXT | UNIQUE, compared without letter case |
| password | TEXT | pbkdf2$200000$salt$hash |

| Attribute | Data Type | Constraint |
|---|---|---|
| role | TEXT | NOT NULL, default 'Customer' |
| phone | TEXT | UAE mobile number |
| currency | TEXT | AED, USD, EUR or GBP (default AED) |
| created_at | TEXT | sign-up time |
| banned / banned_at / banned_by | INTEGER / TEXT / TEXT | banned is 0 or 1 |
| ssn_hash / ssn_last4 | TEXT | salted hash and last 4 digits |
| venue_id | INTEGER | references venues(id), staff only |
| manager_email | TEXT | staff only, just stored |

*Table 2: The users table holds one row per account. The SSN columns are only filled for non-customer roles, and the venue columns only for staff.*

## Table Name: venues

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| name / address | TEXT | name NOT NULL |
| vendor_email | TEXT | owner |
| maps_url | TEXT | Google Maps https link or '' |

*Table 3: The venues table holds the places where events happen and the vendor that owns them.*

## Table Name: events

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| title | TEXT | NOT NULL |
| category / price / description | TEXT | |
| vendor_email | TEXT | owner |
| venue_id | INTEGER | FOREIGN KEY venues(id) |
| event_datetime | TEXT | 'YYYY-MM-DD HH:MM' for one-time events |
| is_recurring / recur_day / recur_time | TEXT | 'Recurring' or 'One-Time', weekdays, time |
| capacity | INTEGER | NOT NULL default 20, from 1 to 5000 |

*Table 4: The events table holds the experiences that can be booked. Price is text such as '240 AED'.*

## Table Name: bookings

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| customer_email | TEXT | NOT NULL, customer or partner |
| event_id | INTEGER | FOREIGN KEY events(id) |
| event_title / venue_name | TEXT | copies kept for history |

| Attribute | Data Type | Constraint |
|---|---|---|
| booking_date / event_time | TEXT | booking_date NOT NULL |
| guests | INTEGER | NOT NULL default 1 |
| total_price | INTEGER | NOT NULL, AED with VAT |
| ticket_code | TEXT | UNIQUE NOT NULL, 16 digits |
| status | TEXT | 'Confirmed' or 'Cancelled' |
| client_name | TEXT | the partner's client group |
| created_at | TEXT | default CURRENT_TIMESTAMP |
| cancelled_at / cancelled_by / cancelled_by_role / cancel_reason | TEXT | cancellation record |
| refund_percent / refund_amount | INTEGER / REAL | 100 or 50, amount in AED |

*Table 5: The bookings table holds every paid booking and, when it is cancelled, who did it, why and how much was refunded.*

## Table Name: tickets

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| ticket_code | TEXT | UNIQUE |
| event_title / customer_email | TEXT | |
| status | TEXT | 'Valid', 'Used' or 'Cancelled' |

*Table 6: The tickets table holds the entry tickets that staff check in.*

## Table Name: reviews

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| booking_id | INTEGER | NOT NULL UNIQUE, FOREIGN KEY bookings(id) |
| event_id / customer_email | INTEGER / TEXT | index on event_id |
| rating | REAL | 0 to 5 in steps of 0.5 |
| comment | TEXT | up to 500 characters |
| created_at / updated_at | TEXT | default CURRENT_TIMESTAMP |

*Table 7: The reviews table holds one rating per booking. A second review of the same booking replaces the first.*

## Table Name: support_tickets

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| customer_email / subject / message | TEXT | NOT NULL |
| event_title | TEXT | the event it is about |
| status | TEXT | 'Open' or 'Resolved' |
| created_at / resolved_by / resolved_at | TEXT | |

*Table 8: The support_tickets table holds questions from customers about an event.*

## Table Name: reports

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| reporter_email / reporter_role | TEXT | NOT NULL |
| category / offender_type / offender_email | TEXT | what happened and who it is about |
| subject / description | TEXT | NOT NULL |
| status / resolution | TEXT | 'Open' or 'Resolved' |
| created_at / resolved_by / resolved_at | TEXT | |

*Table 9: The reports table holds the problems that users report to the admin.*

## Table Name: vehicle_types

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| name | TEXT | UNIQUE NOT NULL |
| capacity | INTEGER | NOT NULL |

*Table 10: The vehicle_types table says how many seats each kind of vehicle has (desert buggy 2, 4x4 Land Cruiser 6, Tesla sedan 4, mini-yacht 20, RTA bus 50).*

## Table Name: vehicles

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| license_number | TEXT | UNIQUE NOT NULL |
| vehicle_type / location | TEXT | NOT NULL |
| status | TEXT | 'Available', 'Assigned' or 'Maintenance' |
| dispatcher_email | TEXT | owner |
| auto_reassign_recurring | INTEGER | 0 or 1 |

*Table 11: The vehicles table holds the fleet of each dispatcher.*

## Table Name: event_vehicles

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| event_id / vehicle_id | INTEGER | FOREIGN KEYs |
| status | TEXT | 'Assigned' or 'Concluded' |

*Table 12: The event_vehicles table links vehicles to the events they serve.*

### Table Name: partner_cart

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| partner_email | TEXT | NOT NULL, indexed |
| event_id | INTEGER | NOT NULL, FOREIGN KEY events(id) |
| booking_date / guests / client_name | TEXT / INTEGER / TEXT | |
| created_at | TEXT | default CURRENT_TIMESTAMP |

*Table 13: The partner_cart table is the partner's shopping list.*

### Table Name: vendor_categories

| Attribute | Data Type | Constraint |
|---|---|---|
| id | INTEGER | PRIMARY KEY AUTOINCREMENT |
| vendor_email / name | TEXT | NOT NULL, no repeated name per vendor |

*Table 14: The vendor_categories table holds the categories a vendor can pick for events.*

We also added four indexes for the queries that run most often: bookings by customer, bookings by event and date, the cart by partner and reviews by event.

Section 3

# 3.0 Architectural and component-level design

**Suitable architecture style:** layered architecture, split over a client and a server (three tiers).

**Reasoning:** We have six roles that all work on the same seats and the same bookings. If each screen had its own copy of the rules, sooner or later two screens would disagree, for example about how many seats are left. Putting the rules in one server layer, under all the screens, avoids that. Layers also let us work in parallel. One of us could change a screen while another changed a SQL query, and as long as the JSON stayed the same nothing broke. The cost is an extra step for every action, because the client has to call the server even for small things. On one machine this delay is too small to notice.

Our layered architecture has three layers:

**Presentation layer (Layer 2):** the Tkinter screens in the src folder. navigate.py switches between the login, sign-up and password screens, and routing.py opens the home screen that matches the user's role. Every home screen extends BaseWelcomeScreen, which draws the dark side menu, the top bar with the clock, and the Report, Settings and Log Out buttons.

**Application layer (Layer 1):** server.py, a Flask REST API. Each endpoint reads the JSON body, checks the caller with a small guard called _blocked (unknown or banned accounts and wrong roles get HTTP 403), calls one data function and returns JSON.

**Data layer (Layer 0):** database_core.py and database_events.py, which hold all SQL and the business rules: password hashing, SSN checks, seats, VAT, ticket codes, refunds, venue scope and bans.

A good example of why this matters is the seat check. The event page checks the remaining seats before it opens the payment window, but that is only for the user's comfort. The real check happens again in the data layer inside a BEGIN IMMEDIATE transaction, when the booking is saved. That is how we prevent double booking. In

Phase 1 we had planned a 15-minute seat lock instead, but the transaction turned out to be simpler and safer.

## 3.1 System Structure

The client is organised by screen and the server by data area. The welcome package in the client has one module per role and a few shared views: ReportView, VehicleView, DispatchView, the cancellation helpers and the shared table style. The server has the API module and the two data modules.

### 3.1.1 Architecture diagram

The full class diagram did not fit on one page, so we split it in two. Figure 3 shows the home screens and how they inherit from BaseWelcomeScreen. Figure 4 shows the modules behind them and the server. Python modules that are not classes are marked with <<module>>.

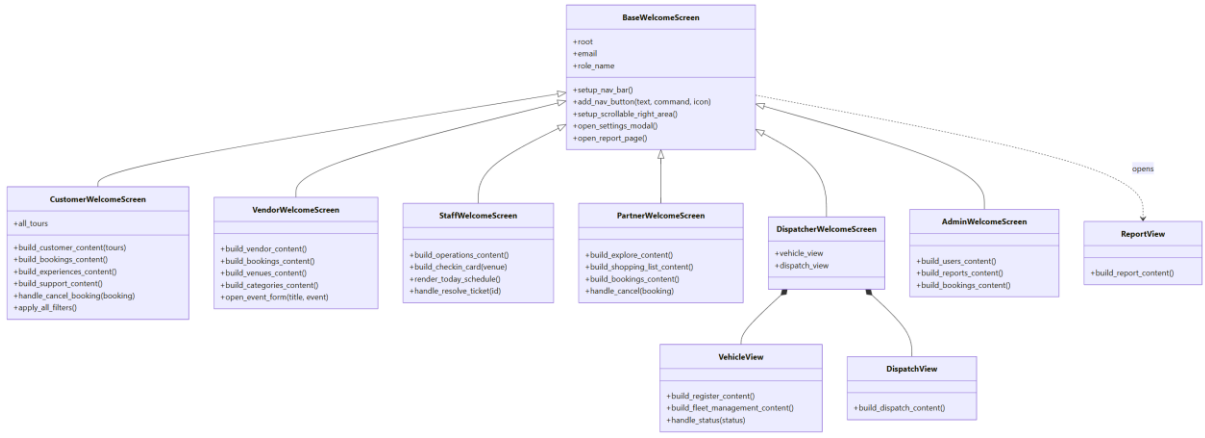



*Figure 3: Class diagram, part 1: the home screens and shared views*

Figure 3: every role gets its own subclass of BaseWelcomeScreen, so the menu, top bar and settings look the same everywhere and each subclass only adds its own pages. The dispatcher screen is the only one that is built from two helper views, because its fleet and dispatch pages are large.

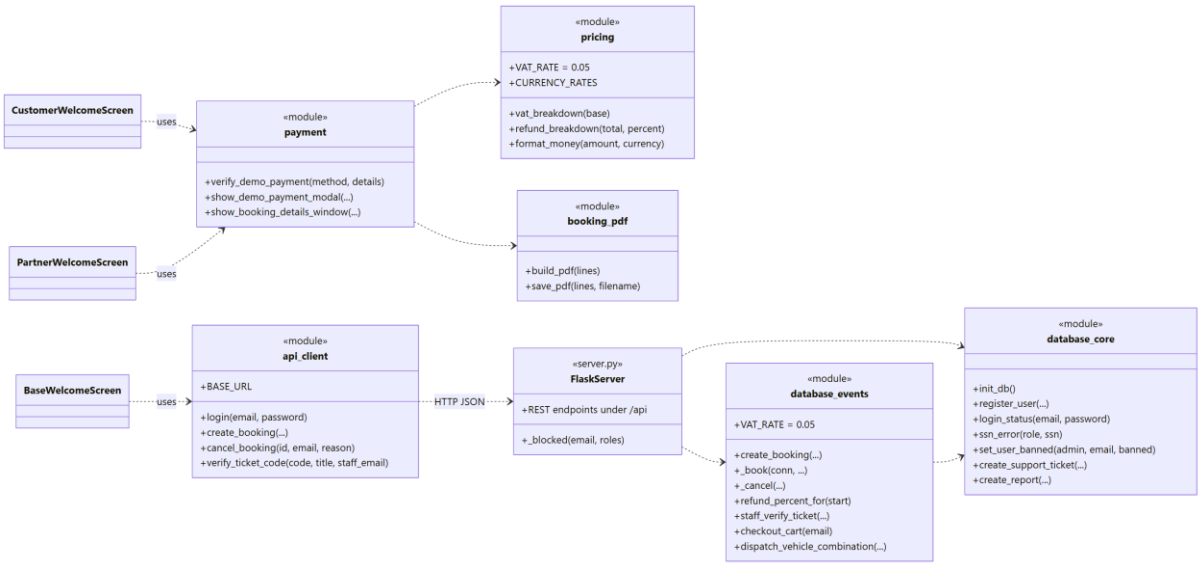



*Figure 4: Class diagram, part 2: client modules, Flask server and data modules*

Figure 4: the screens only know api_client. The payment window uses pricing for VAT and currency and booking_pdf for the receipt. On the server side, server.py calls the two data modules.

Figures 5 to 7 are the use case diagrams. We drew two roles per diagram to keep them readable. The components in section 3.2 follow these use cases.

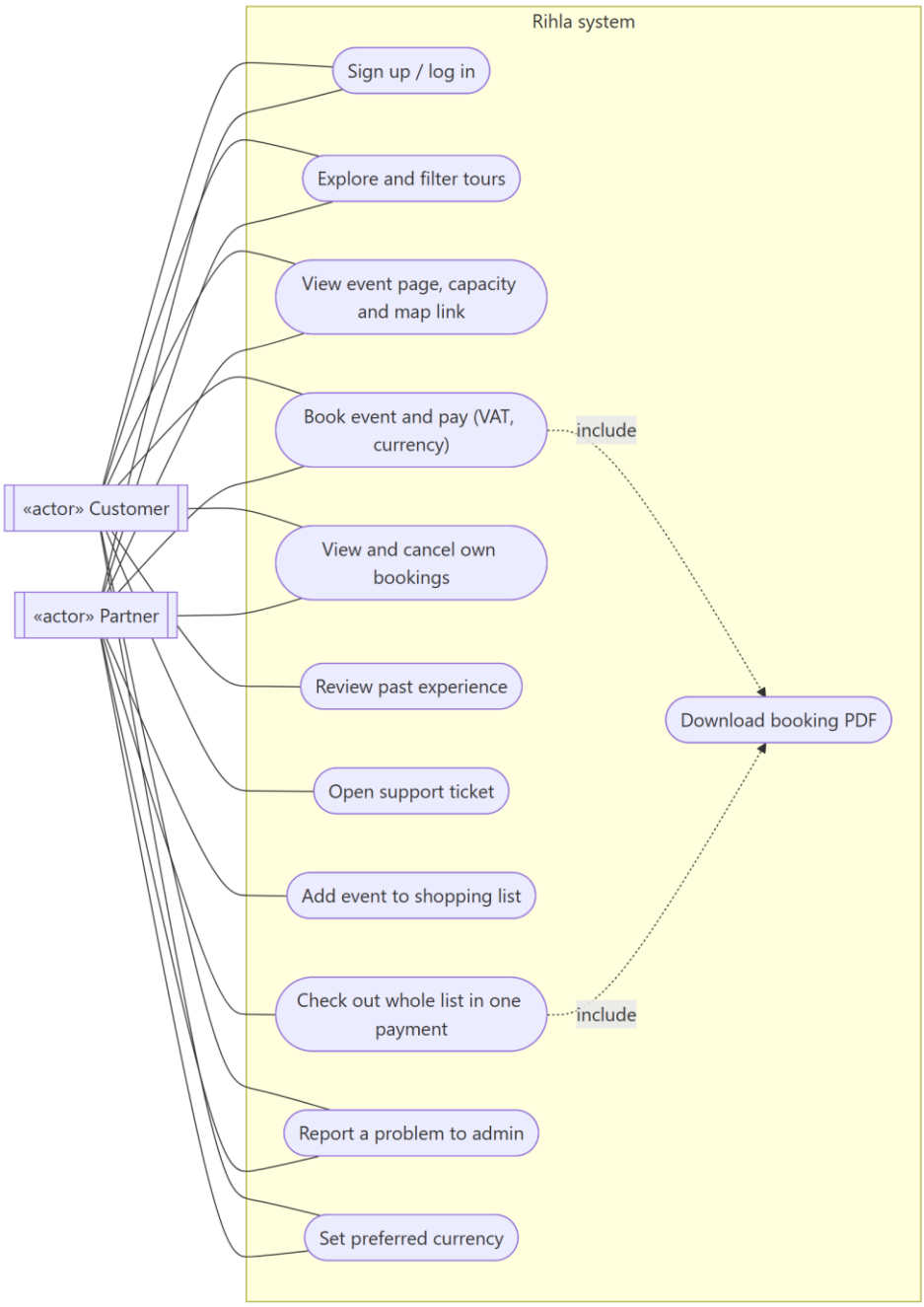



*Figure 5: Use cases of the Customer and the Partner*

Customers and partners share most of their use cases. The partner has two extra ones: the shopping list and paying for the whole list at once. Both paths end with the Booking Details window, where the PDF can be downloaded.

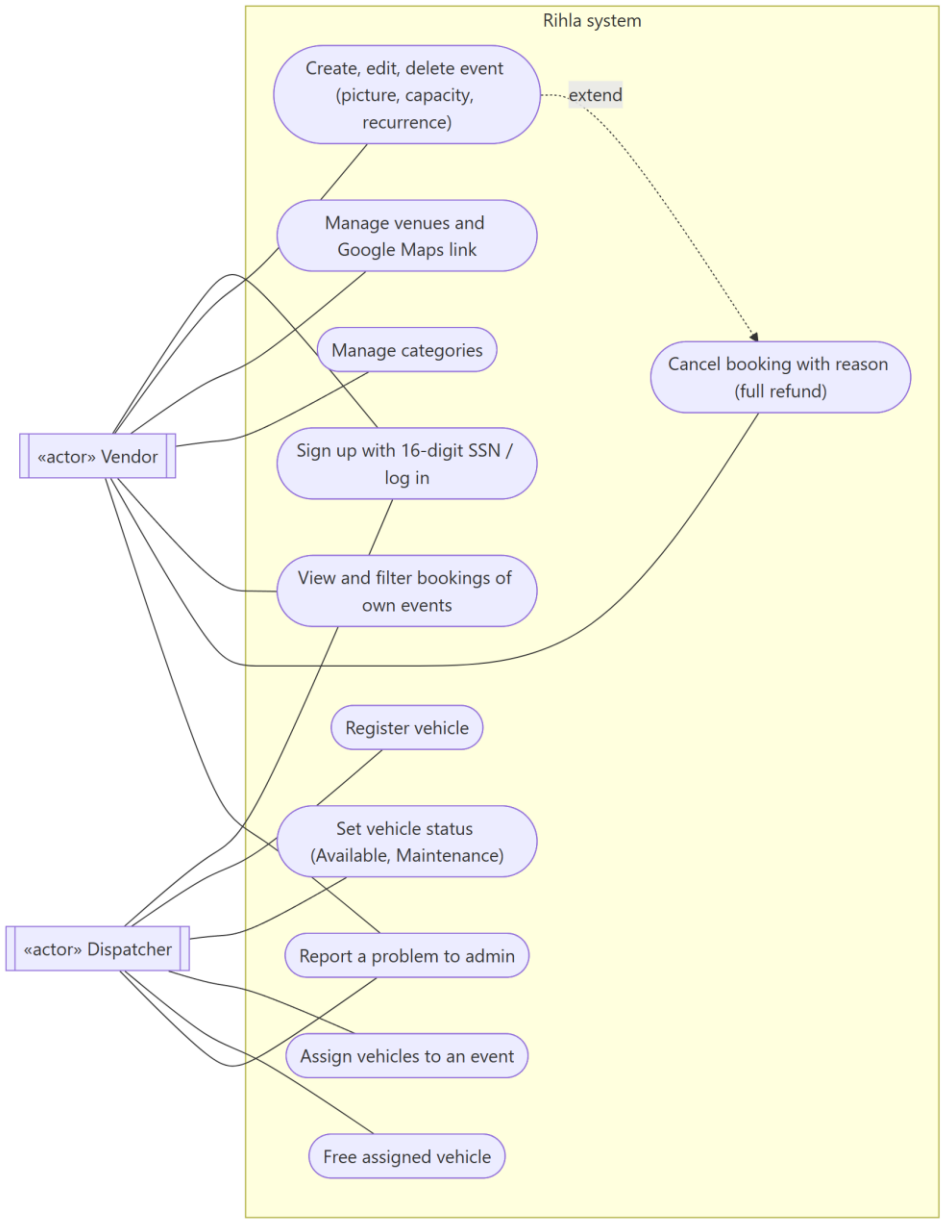



*Figure 6: Use cases of the Vendor and the Dispatcher*

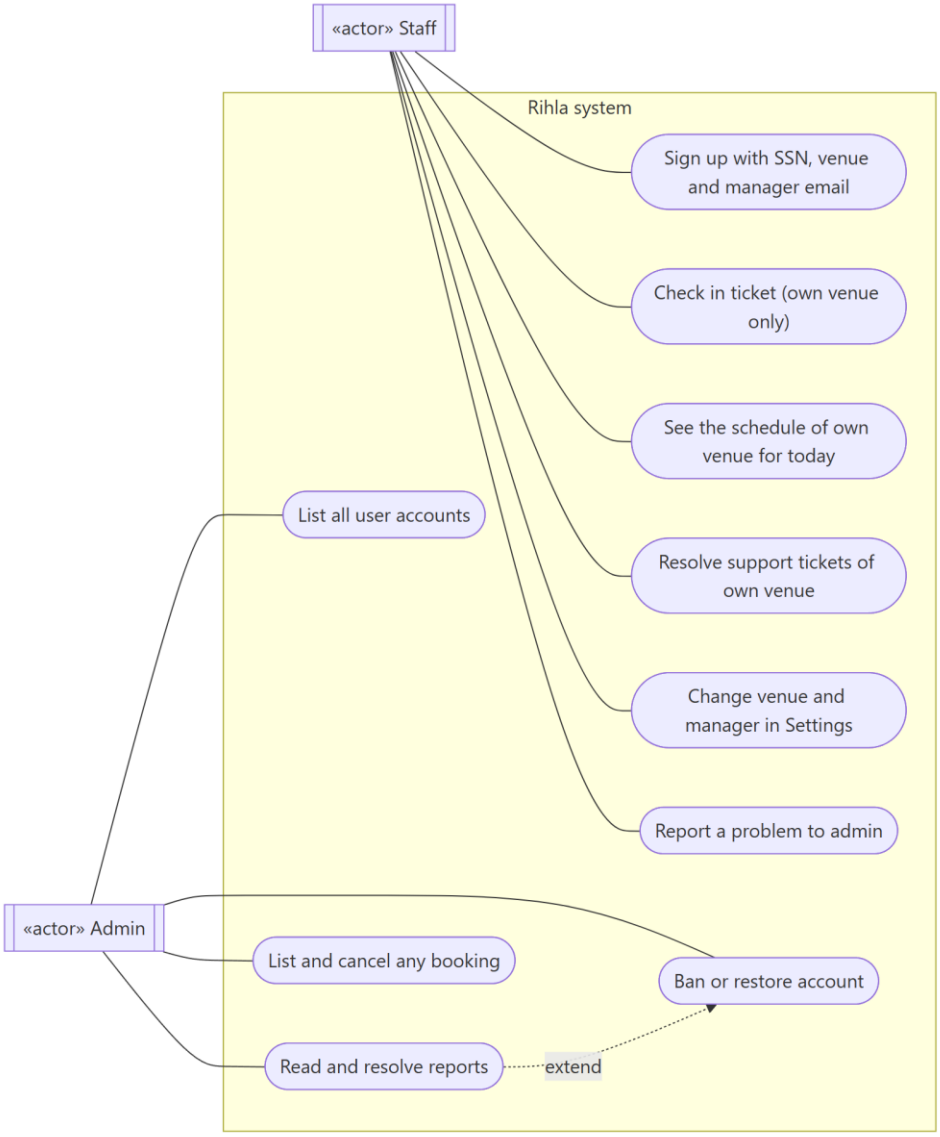



*Figure 7: Use cases of the Staff and the Admin*

The admin does not have a Report button, because admins are the ones who read reports. Banning an account is often done straight from a report, which is why it is drawn as an extension of it.

## 3.2 Description for Components

We split Rihla into seven components. Each one is described with the template headings. The processing detail is written as a simple Start and End flow, and each operation is given with its Python name so it can be found in the code.

## 3.2 Description for Component “Accounts and Sign-in”

### 3.2.1 Processing narrative (PSPEC) for component

This component covers everything about who the user is. It creates accounts, signs people in and out, resets passwords and keeps each user's preferred currency. The login screen has quick buttons for the six demo accounts, which saved us a lot of typing during testing.

Sign-up has an account type picker with five roles. Admin is not one of them, so nobody can make an admin account from the app. Customers sign up with email, phone and password. Every other role must also give a 16-digit SSN, and staff must pick the venue they work at and the email of its manager. We store the SSN the same way as a password: only a salted hash, plus the last 4 digits.

Passwords are hashed with PBKDF2 and 200,000 rounds. Some very early test accounts still had plain text passwords, so the login quietly upgrades such a password to a hash the first time it is used.

### 3.2.2 Component interface description

The login screen is the first thing every user sees. From there the user can open Create Account or Forgot Password. After a correct login the client asks the server for the role and opens the matching home screen. A banned user gets a clear message instead.

| Endpoint | Sends | Gets back |
|---|---|---|
| POST /api/auth/login | email, password | stored email and role, or 401 / 403 |
| POST /api/users/register | email, password, role, phone, ssn, venue_id, manager_email | success or the reason |
| PUT /api/users/password | email, new_password | success |
| GET and PUT /api/users/currency | email, currency | currency code |
| GET and PUT /api/staff/venue | email, venue_id, manager_email | venue and manager |

*Table 15: Server endpoints used by “Accounts and Sign-in”*

### 3.2.3 Component processing detail

- Start
- User opens the app and sees the login screen
- User chooses to log in, sign up or reset the password

- If signing up, user picks an account type and fills in the form (SSN for non-customers, venue and manager email for staff)
- User verifies the email and phone with the code 000000
- System checks the details and saves the account
- If logging in, system checks the email and password
- Based on the result, user enters the home screen of their role, or sees an error
- User can change the currency or password from Settings and log out at any time
- End

#### 3.2.3.1 Design Class hierarchy for component Accounts and Sign-in

On the client the component is made of screen functions rather than classes: show_login_screen, show_create_account_screen, show_forgot_password_screen, show_reset_password_window and open_social_login. The SSN is typed into DigitBoxes, a small widget we wrote that shows one box per digit. On the server the work is done by register_user, login_status, ssn_error, staff_venue_error, hash_password and verify_password in database_core.py.

#### 3.2.3.2 Restrictions/limitations for component Accounts and Sign-in

There is no session token. After login the client only remembers the email and sends it with later requests. The email and phone codes are always 000000, and the Google and Apple buttons only open an ordinary email and password form. The password change endpoint does not ask for the old password.

#### 3.2.3.3 Performance issues for component Accounts and Sign-in

Hashing with 200,000 rounds is slow on purpose, so a login takes a moment. It happens once per login, so it does not slow down the rest of the app.

#### 3.2.3.4 Design constraints for component Accounts and Sign-in

Passwords need at least 8 characters. The SSN must be exactly 16 digits and cannot be the same digit 16 times. The phone number is a UAE mobile number with 9 digits after +971.

#### 3.2.3.5 Processing detail for each operation of component Accounts and Sign-in

The component has the operations listed below:

```
+ login_status(email: str, password: str): str
+ register_user(email, password, role, phone, ssn, venue_id,
manager_email): bool
+ ssn_error(role: str, ssn: str): str or None
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. login_status()** is invoked when the user presses Sign In. It returns 'ok', 'invalid' or 'banned'.

**2. register_user()** is invoked when the user presses Create Account. It saves the account only if every rule passes.

**3. ssn_error()** is invoked by sign-up on both the client and the server. It returns the message to show, or nothing when the SSN is fine.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. login_status()**

```
Start
Find the account by email, ignoring letter case
If there is no account or the password does not match, return 'invalid'
If the stored password is old plain text, replace it with a hash
If the account is banned, return 'banned'
Return 'ok'
End
```

**2. register_user()**

```
Start
If the role is not allowed at sign-up, use 'Customer'
If the email is taken or the SSN is wrong for this role, return false
If the role is Staff, check the venue and manager email
Save the account with the hashed password and hashed SSN (last 4 digits
kept)
Return true
End
```

### 3. ssn_error()

```
Start
If the role is Customer, return nothing
Remove spaces and dashes
If the SSN is empty, not 16 digits, or one digit repeated, return the
message
Return nothing
End
```

## 3.2 Description for Component “Events and Venues”

### 3.2.1 Processing narrative (PSPEC) for component

This is the vendor's side of Rihla. A vendor creates events with a title, category, price in AED, description, venue, a time, a capacity and a picture. An event either happens once on a date or repeats on chosen weekdays, for example every Tuesday and Thursday at 15:30. The vendor also keeps a list of venues, each with an optional Google Maps link, and a list of categories.

Deleting an event is the most careful operation here, because customers may already have paid. The server frees the event's vehicles, cancels its upcoming bookings that were not used yet with a full refund, cancels their tickets, and only then deletes the event and its picture.

### 3.2.2 Component interface description

The vendor works from the pages My Events, Bookings, Manage Venues, Manage Categories and Add New Event. One form is used both for adding and for editing an event. Pictures are picked with the normal file dialog and sent with the rest of the form.

| Endpoint | Sends | Gets back |
|---|---|---|
| GET /api/events | | all events |
| POST /api/events | event fields, picture (base64) | success (vendors only) |
| PUT and DELETE /api/events/<id> | event fields, or nothing | success |
| POST /api/venues, DELETE /api/venues/<id> | name, address, maps_url | success or 'in use' |
| PUT /api/venues/<id>/maps_url | vendor_email, maps_url | cleaned link or error |
| GET, POST, DELETE /api/vendor_categories | vendor_email, name | categories |

*Table 16: Server endpoints used by “Events and Venues”*

### 3.2.3 Component processing detail

- Start
- Vendor opens My Events
- Vendor adds, edits or deletes an event, a venue or a category
- System checks the capacity, the picture and the map link
- System saves the change and renames the event in old bookings and tickets if the title changed
- If an event is deleted, system cancels its upcoming bookings with a full refund
- The updated event appears for customers and partners
- End

#### 3.2.3.1 Design Class hierarchy for component Events and Venues

VendorWelcomeScreen builds all five pages, with open_event_form for both adding and editing. event_images.py loads, shrinks and shows pictures. On the server the work is in create_event, update_event, delete_event, create_venue, delete_venue, validate_maps_url and update_venue_maps_url.

#### 3.2.3.2 Restrictions/limitations for component Events and Venues

Updating or deleting an event and deleting a venue do not check that the caller owns it. The client only shows a vendor their own records, but the server does not enforce it yet. Prices are saved as text and only the digits are read, so a price like 99.50 is not possible.

#### 3.2.3.3 Performance issues for component Events and Venues

Pictures are shrunk to 1400 pixels before upload and limited to 4 MB, so the lists with many cards still load quickly.

#### 3.2.3.4 Design constraints for component Events and Venues

Capacity must be between 1 and 5000. Pictures must be JPEG. A venue cannot be deleted while events use it or staff work there. Map links must be https links to Google Maps.

#### 3.2.3.5 Processing detail for each operation of component Events and Venues

The component has the operations listed below:

```
+ delete_event(event_id: int): None
+ validate_maps_url(url: str): (str, str or None)
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. delete_event()** is invoked when the vendor presses Delete on an event card and confirms.

**2. validate_maps_url()** is invoked when a venue is created or its map link is changed. It returns the cleaned link or a message.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. delete_event()**

```
Start
Set the event's assigned vehicles back to Available
Remove the event's vehicle assignments
Cancel upcoming bookings with an unused ticket (100% refund, reason given)
Cancel their tickets
Delete the event
End
```

**2. validate_maps_url()**

```
Start
If the link is empty, accept it
If it is too long or has spaces, refuse it
Add https:// when needed
If the host is a Google Maps address, accept it, otherwise refuse it
End
```

## 3.2 Description for Component “Booking and Payment”

### 3.2.1 Processing narrative (PSPEC) for component

This component turns a choice of event, date and guests into a paid booking with a ticket. It is used by customers and partners.

The event page shows a calendar where only the allowed dates can be picked, a capacity bar such as '12 of 30 remaining' (with 'Hurry up!' when it is almost full) and the total price. The payment window shows the base price, the 5% VAT and the total in the currency the user picks. Visa, Mastercard and bank transfer are offered, already filled in with demo details. When the payment is approved, the server saves the booking and a new 16-digit ticket, and the Booking Details window shows everything with a Download PDF button.

Partners work a little differently. They have no limit of 10 guests per booking, they can type a client name, and they can collect several events in a shopping list and pay for all of them at once.

### 3.2.2 Component interface description

The customer starts from Explore Tours and opens an event with View Details. The partner has the same page with Add to List and Pay Now instead of one booking button. After payment both see the Booking Details window, and the booking appears in My Bookings or Partner Bookings.

| Endpoint | Sends | Gets back |
|---|---|---|
| GET /api/events/<id>/capacity?date= | event id, date | total, booked, remaining |
| POST /api/bookings | email, event_id, booking_date, guests, client_name | booking with base, VAT, total, ticket code |
| GET /api/bookings/customer/<email> | | bookings with status and refund info |
| POST /api/cart, GET /api/cart/<email> | partner_email, event_id, date, guests, client_name | cart items |
| POST /api/cart/<email>/checkout | | all bookings, or none |

*Table 17: Server endpoints used by "Booking and Payment"*

### 3.2.3 Component processing detail

- Start
- User opens an event and picks a date and the number of guests
- System shows the seats left on that date and the price
- User presses Proceed to Booking (or Add to List and later Pay, for partners)
- System shows the payment window with base price, VAT and total in the chosen currency
- User pays with the demo details

- System checks the seats again, saves the booking and creates a ticket code
- User sees the Booking Details window and can download the PDF
- End

### 3.2.3.1 Design Class hierarchy for component Booking and Payment

The event page is show_event_screen in screens/event_screen.py. CustomerWelcomeScreen and PartnerWelcomeScreen list the events and bookings. payment.py holds the payment window, the demo check and the Booking Details window, pricing.py does the VAT and currency maths and booking_pdf.py writes the PDF without any extra library. On the server the main functions are create_booking, _book, _capacity_info, add_vat, _new_ticket_code, add_to_cart and checkout_cart.

The booking flow is the heart of Rihla, so we show it as a sequence diagram here (Figure 8) and the partner's cart checkout in Figure 9.

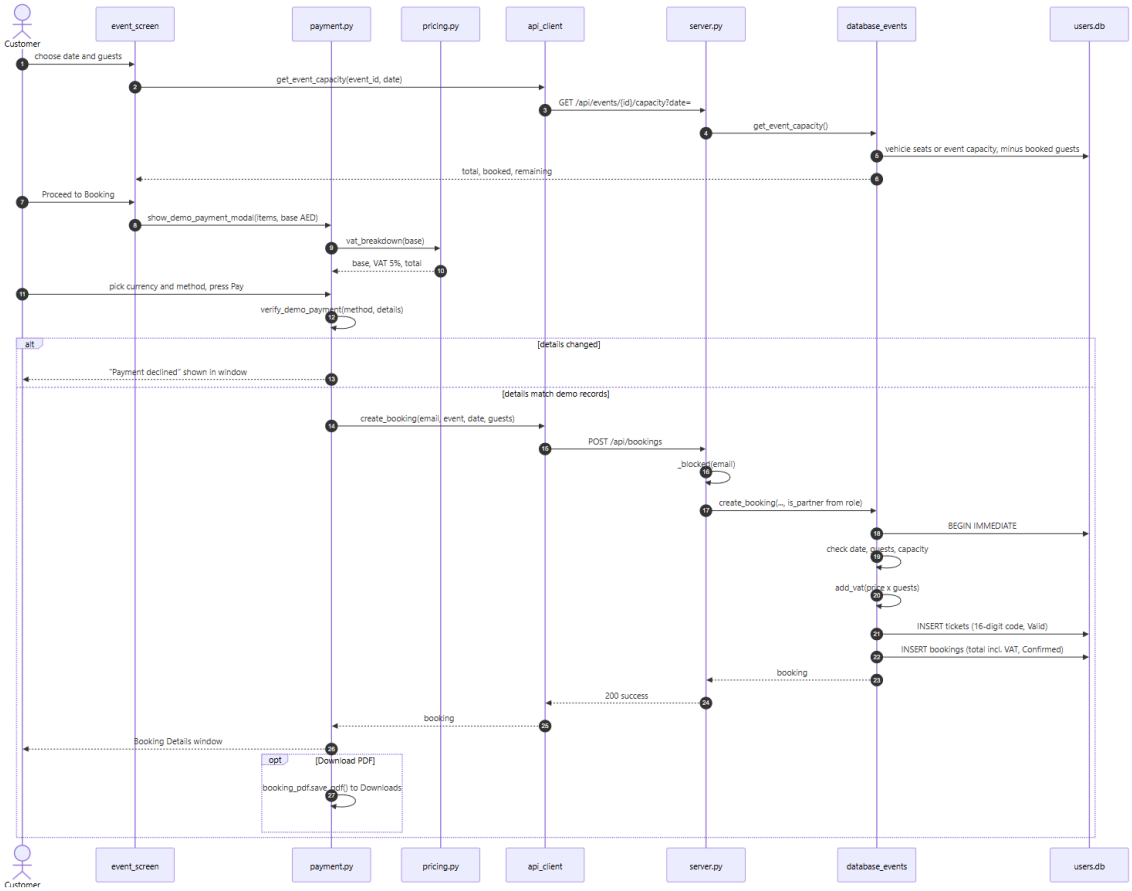



*Figure 8: Sequence diagram: a customer books an event and pays*

Figure 8: the capacity is read first for the chosen date. The payment window adds VAT and shows the chosen currency. Only after the demo payment is approved does the client send the booking, and the server checks the seats again inside its transaction before it creates the ticket.

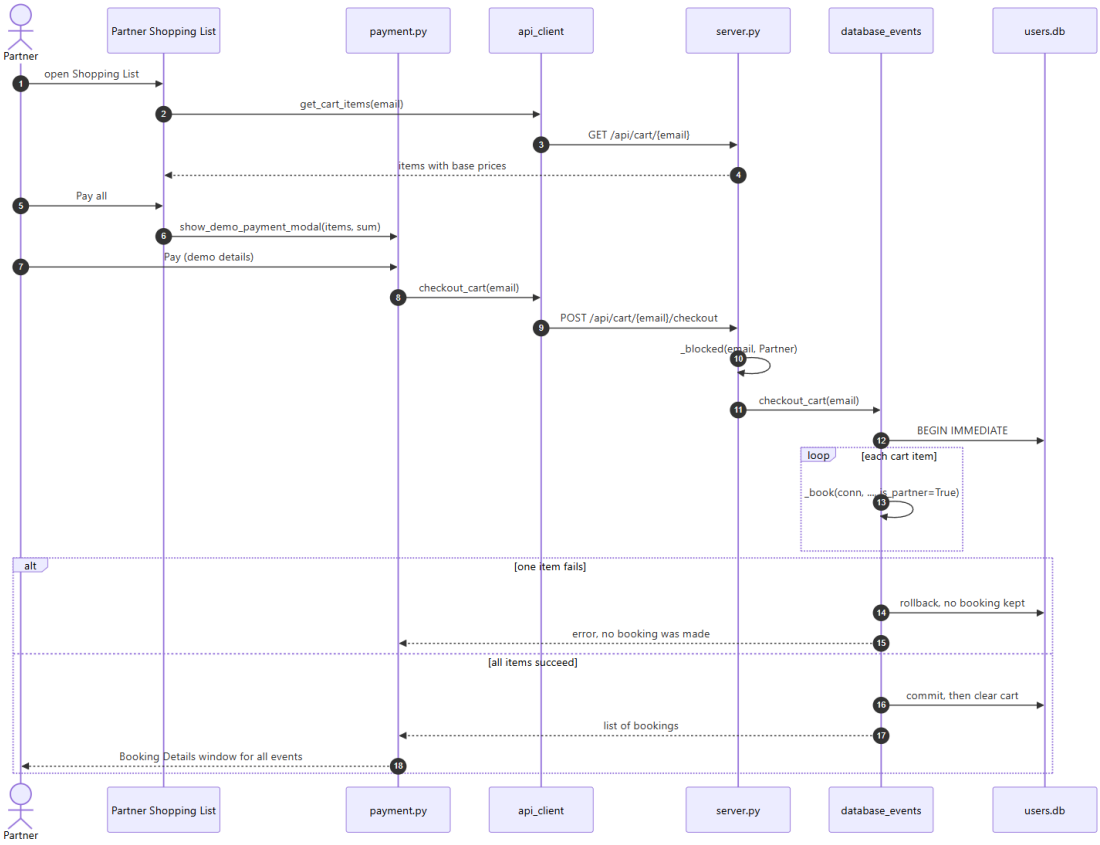



*Figure 9: Sequence diagram: a partner pays for the whole shopping list*

Figure 9: every item in the cart is booked inside one transaction. If one of them fails, for example because the event filled up in the meantime, nothing is booked and the cart stays as it was.

### 3.2.3.2 Restrictions/limitations for component Booking and Payment

The payment is a demo. It only checks that the form still has the fixed demo card or bank details, so changing a digit makes it fail on purpose. Currency rates are fixed numbers and everything is charged in AED. Also, the payment is approved on the client before the server saves the booking. If the booking fails at that point, the Booking Details window says NOT CONFIRMED and asks the user to contact support. With a real payment provider this order would have to change.

#### 3.2.3.3 Performance issues for component Booking and Payment

A booking is one short write. BEGIN IMMEDIATE takes the database write lock at the start, so two people booking the last seats at the same time are handled one after the other instead of both getting them.

#### 3.2.3.4 Design constraints for component Booking and Payment

The date must be today or later and must fit the event: the right weekday for a repeating event or the exact date for a one-time event. Customers book 1 to 10 guests. Nobody can book more than the seats left on that date.

#### 3.2.3.5 Processing detail for each operation of component Booking and Payment

The component has the operations listed below:

```
+ create_booking(email, event_id, booking_date, guests, is_partner,
client_name): dict
- _book(conn, email, event_id, day, guests, is_partner, client_name): dict
- _capacity_info(conn, event_id, day): dict
+ checkout_cart(partner_email: str): dict
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. create_booking()** is invoked after the demo payment is approved. It opens a transaction and calls _book.

**2. _book()** is invoked by create_booking and by checkout_cart. It does the checks and saves the booking inside the transaction it is given.

**3. _capacity_info()** is invoked whenever seats are needed: on the event page, when booking and when adding to the cart.

**4. checkout_cart()** is invoked when the partner pays for the shopping list.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. create_booking()**

```
Start
Open a write transaction
Call _book with the request
Return its result
End
```

### 2. _book()

```
Start
Check the date and number of guests (1 to 10 unless partner)
Refuse dates in the past or dates the event does not run on
Get the seats left on that date
If there are not enough seats, refuse with the numbers
Make a new random 16-digit ticket code that is not used yet
Work out base = price x guests, VAT = 5% of base, total = base + VAT
Save the ticket as Valid and the booking as Confirmed with the total
Return the booking with base, VAT and total
End
```

### 3. _capacity_info()

```
Start
If the event has assigned vehicles, total = sum of their seats
Else total = the event's own capacity
Booked = guests of confirmed bookings on that day
Remaining = total - booked (never below 0)
End
```

### 4. checkout_cart()

```
Start
If the cart is empty, refuse
Open a write transaction
Book every item with _book
If one item fails, undo everything and refuse
Empty the cart and return all bookings
End
```

## 3.2 Description for Component “Cancellations and Refunds”

### 3.2.1 Processing narrative (PSPEC) for component

We wanted one refund policy that every role follows, written in one function. A customer or partner can cancel their own booking. They get 100% back if the event starts in 48 hours or more, 50% between 48 and 24 hours, and they cannot cancel in the last 24 hours. VAT is refunded in the same share as the price.

A vendor can cancel bookings on their own events, and the admin can cancel any booking. Both can do it right up to the start, always refund 100%, and must give the customer a reason. Every cancellation saves when it happened, who did it, their role, the reason, the percent and the amount. The seats are free again straight away.

### 3.2.2 Component interface description

Customers cancel from My Bookings and partners from Partner Bookings. Each card already shows what the refund would be, so there are no surprises. Vendors and admins select a row in their bookings table and press Cancel, and a small window asks for the reason.

| Endpoint | Sends | Gets back |
|---|---|---|
| POST /api/bookings/<id>/cancel | customer_email | refund percent, base, VAT, total |
| POST /api/bookings/<id>/vendor_cancel | vendor_email, reason | refund (vendors only) |
| POST /api/admin/bookings/<id>/cancel | admin_email, reason | refund (admins only) |

*Table 18: Server endpoints used by "Cancellations and Refunds"*

### 3.2.3 Component processing detail

- Start
- User presses Cancel on a booking
- System shows the refund the user would get (or asks vendors and admins for a reason)
- User confirms
- System checks that the user may cancel this booking and that the event has not started
- System works out the refund percent from the hours left
- System marks the ticket and the booking as cancelled and saves the refund
- User sees the refund amount and the seats become free
- End

#### 3.2.3.1 Design Class hierarchy for component Cancellations and Refunds

The client part is small: cancellation.py has the policy text, the confirm dialog and the reason dialog, and pricing.refund_breakdown shows the numbers. On the

server, cancel_booking, vendor_cancel_booking and admin_cancel_booking all call one shared function, _cancel, and only pass a different owner check.

Figure 10 shows a customer cancelling, and Figure 11 shows the refund decision as an activity diagram.

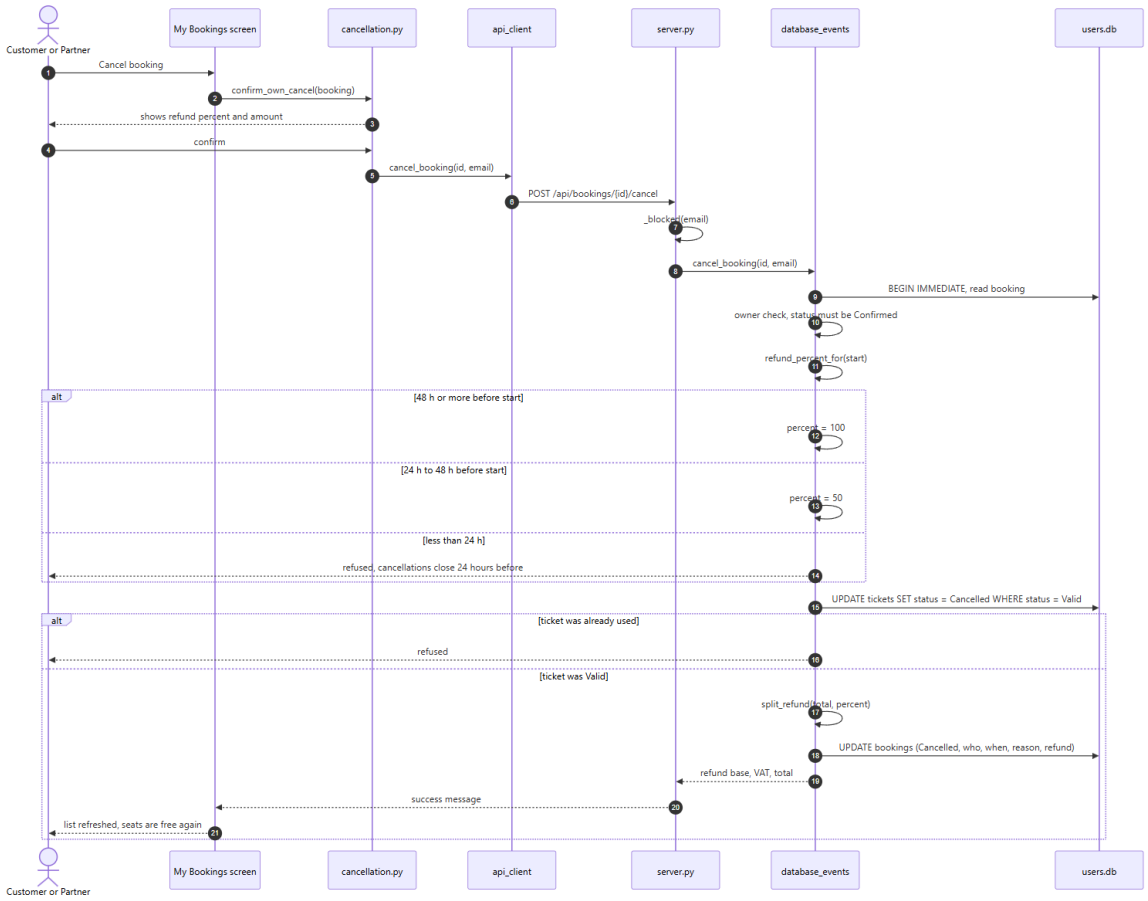



*Figure 10: Sequence diagram: a booker cancels and gets a refund*

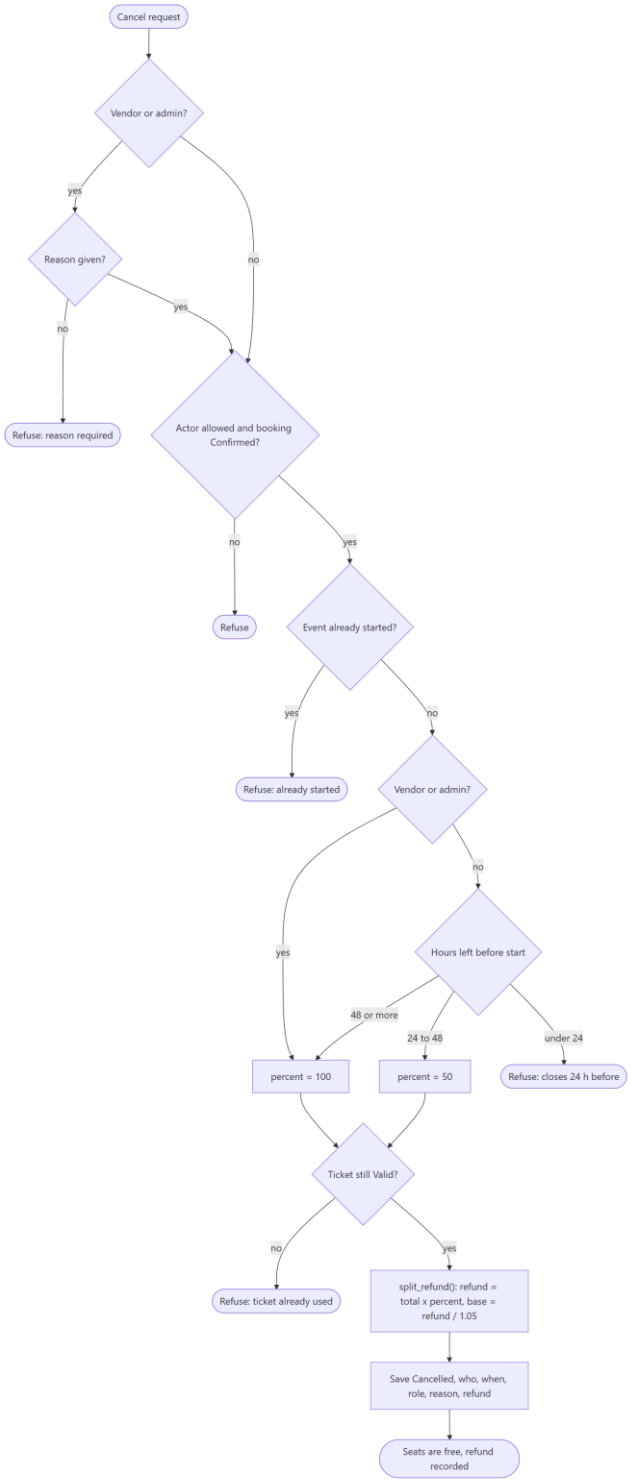



*Figure 11: Activity diagram of the refund decision*

Figure 11: the same function handles customers, partners, vendors and admins. The only differences are who is allowed to cancel the booking and whether the refund follows the hours left or is always 100%.

#### 3.2.3.2 Restrictions/limitations for component Cancellations and Refunds

Refunds are written to the database only. No money moves, because the payments are a demo too. A booking with no time saved counts from midnight of its date.

#### 3.2.3.3 Performance issues for component Cancellations and Refunds

Each cancellation is one short transaction, the same as a booking.

#### 3.2.3.4 Design constraints for component Cancellations and Refunds

A checked-in ticket cannot be cancelled, and nobody can cancel after the event has started. The reason can be at most 300 characters.

#### 3.2.3.5 Processing detail for each operation of component Cancellations and Refunds

The component has the operations listed below:

```
- _cancel(booking_id, actor_email, actor_role, reason, owner_check,
by_operator): dict
+ refund_percent_for(start: datetime): int or None
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. _cancel()** is invoked by all three cancel endpoints.

**2. refund_percent_for()** is invoked by _cancel and also when bookings are listed, so the screen can show the refund in advance.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. _cancel()**

```
Start
If a vendor or admin gives no reason, refuse
Open a write transaction and read the booking
If the actor may not cancel it, or it is not Confirmed, refuse
If the event already started, refuse
Percent = 100 for vendor or admin, otherwise from the hours left
```

```
If the percent is none (under 24 hours), refuse
Mark the ticket Cancelled only if it is still Valid, otherwise refuse
Split the refund into base and VAT
Save the booking as Cancelled with who, when, why and the refund
End
```

**2. refund_percent_for()**

```
Start
Hours left = start time - now
If 48 or more, return 100
If 24 or more, return 50
Return none
End
```

## 3.2 Description for Component “Venue Operations”

### 3.2.1 Processing narrative (PSPEC) for component

This component is for staff on the ground. Every staff account belongs to one venue, chosen at sign-up and changeable in Settings. The Venue Operations page shows that venue, its manager's email, a check-in card and today's schedule for the venue with the number of bookings, guests and guests already checked in.

We added the venue rule late in the project, after we noticed that any staff member could check in any ticket in the city. Now the server refuses a ticket for an event at another venue, the schedule only lists the staff member's own venue, and the Support Tickets page only shows questions about events there. Old support tickets from before customers had to pick an event are shown to all staff.

### 3.2.2 Component interface description

Staff type the 16-digit code into one box per digit and press Validate. The answer appears under the boxes, for example 'Access granted! Ticket valid for ...' followed by how many guests to admit.

| Endpoint | Sends | Gets back |
|---|---|---|
| POST /api/tickets/verify | staff_email, ticket_code | access granted or the reason |
| GET /api/events/today?staff_email= | | today's events at the venue |
| GET /api/support/open?staff_email= | | open questions for the venue |
| POST /api/support/<id>/resolve | staff_email | resolved or refused |

*Table 19: Server endpoints used by “Venue Operations”*

### 3.2.3 Component processing detail

- Start
- Staff member opens Venue Operations
- System shows the venue, the manager and today's schedule
- Staff member types a ticket code and presses Validate
- System checks that the ticket exists, belongs to this venue and is still Valid
- System marks the ticket as Used and says how many guests to admit
- Staff member can open Support Tickets and mark questions as resolved
- End

#### 3.2.3.1 Design Class hierarchy for component Venue Operations

StaffWelcomeScreen builds both pages (build_operations_content, build_checkin_card, render_today_schedule and build_support_content) and reuses DigitBoxes for the code. On the server, staff_verify_ticket finds the venue and calls verify_ticket_code, while get_staff_schedule and get_staff_support_tickets apply the same venue filter.

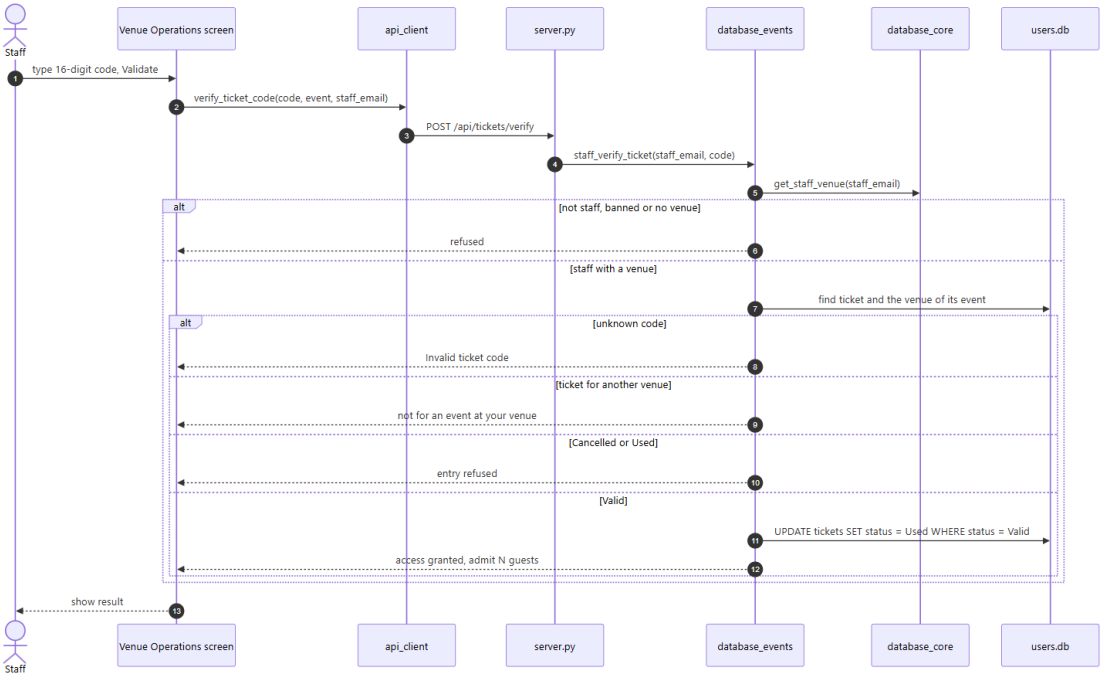



*Figure 12: Sequence diagram: staff check a ticket in*

Figure 12: the server finds the staff member's venue itself, from the email, so the client cannot choose another venue. A ticket for a different venue is refused without saying anything else about it.

#### 3.2.3.2 Restrictions/limitations for component Venue Operations

Codes are typed by hand. There is no QR scanner, and the schedule only covers today.

#### 3.2.3.3 Performance issues for component Venue Operations

The schedule runs two small queries per event of the venue. One venue only has a few events a day, so this is not a problem.

#### 3.2.3.4 Design constraints for component Venue Operations

The update to Used only happens if the ticket is still Valid, so two staff members can never admit the same ticket, even at the same second.

#### 3.2.3.5 Processing detail for each operation of component Venue Operations

The component has the operations listed below:

```
+ staff_verify_ticket(staff_email: str, ticket_code: str): dict
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. staff_verify_ticket()** is invoked when staff press Validate.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. staff_verify_ticket()**

```
Start
Find the venue of this staff member (not banned)
If there is none, refuse
Find the ticket, refuse if it does not exist
If the ticket's event is at another venue, refuse
If the ticket is Cancelled or Used, refuse
Mark the ticket Used if it is still Valid
Return 'Access granted' with the number of guests
End
```

## 3.2 Description for Component “Fleet and Dispatch”

### 3.2.1 Processing narrative (PSPEC) for component

The dispatcher looks after the vehicles. They register vehicles with a license number, a type and a location, keep their status up to date, and give a group of vehicles to an event.

This is also where an event's seats come from. As soon as an event has vehicles, its capacity is the sum of their seats. For example two 4x4 Land Cruisers with 6 seats each give 12 seats, whatever the vendor typed. When the vehicles are freed, the event goes back to the vendor's number.

### 3.2.2 Component interface description

The dispatcher has three pages: Register Vehicles, Fleet Maintenance (a table with filters on the columns) and Event Dispatch, where they pick an event, filter the available vehicles by type and location, and assign the ones they tick.

| Endpoint | Sends | Gets back |
|---|---|---|
| GET /api/vehicles/<dispatcher_email> | | the dispatcher's vehicles |
| POST /api/vehicles, DELETE /api/vehicles/<id> | license_number, vehicle_type, location | success |
| PUT /api/vehicles/<id>/status, POST .../free | status | success |
| POST /api/dispatch | event_id, chosen vehicles | how many were assigned |
| GET /api/dispatch/<event_id>/vehicles | | vehicles assigned to the event |

*Table 20: Server endpoints used by “Fleet and Dispatch”*

### 3.2.3 Component processing detail

- Start
- Dispatcher registers vehicles
- Dispatcher opens Event Dispatch and picks an event
- System lists the available vehicles and the ones already assigned
- Dispatcher ticks vehicles and assigns them
- System marks them Assigned and the event's capacity becomes their seats
- Later the dispatcher frees a vehicle or sets it to Maintenance, which ends the assignment
- End

#### 3.2.3.1 Design Class hierarchy for component Fleet and Dispatch

DispatcherWelcomeScreen is built from two helper views: VehicleView for registering and maintenance and DispatchView for assigning. Both use our shared table style. On the server the functions are create_vehicle, update_vehicle_status, free_vehicle, dispatch_vehicle_combination and get_dispatches_by_event.

#### 3.2.3.2 Restrictions/limitations for component Fleet and Dispatch

This is the weakest part of the server when it comes to security. The vehicle and dispatch endpoints do not check the caller's role or that the vehicle belongs to them. A vehicle is given to an event as a whole, not to one date. The auto_reassign_recurring flag is saved and shown, but nothing uses it yet.

#### 3.2.3.3 Performance issues for component Fleet and Dispatch

Seats are worked out with one grouped query, so it stays quick with a large fleet.

#### 3.2.3.4 Design constraints for component Fleet and Dispatch

Only vehicles that are Available can be assigned. The update has this condition in it, so the same vehicle can never be assigned twice.

#### 3.2.3.5 Processing detail for each operation of component Fleet and Dispatch

The component has the operations listed below:

```
+ dispatch_vehicle_combination(event_id: int, assignments: list): int
+ update_vehicle_status(vehicle_id: int, status: str): None
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. dispatch_vehicle_combination()** is invoked when the dispatcher presses Assign. It returns how many vehicles were really assigned.

**2. update_vehicle_status()** is invoked by Set Maintenance, Set Available and Free.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. dispatch_vehicle_combination()**

```
Start
For each chosen vehicle
```

```
   Set it to Assigned only if it is still Available
   If that worked, link it to the event and count it
Return the count
End
```

### 2. update_vehicle_status()

```
Start
If the new status is not Assigned, end the vehicle's event assignment
Save the new status
End
```

The life of a vehicle is short enough to show in one state diagram (Figure 13).

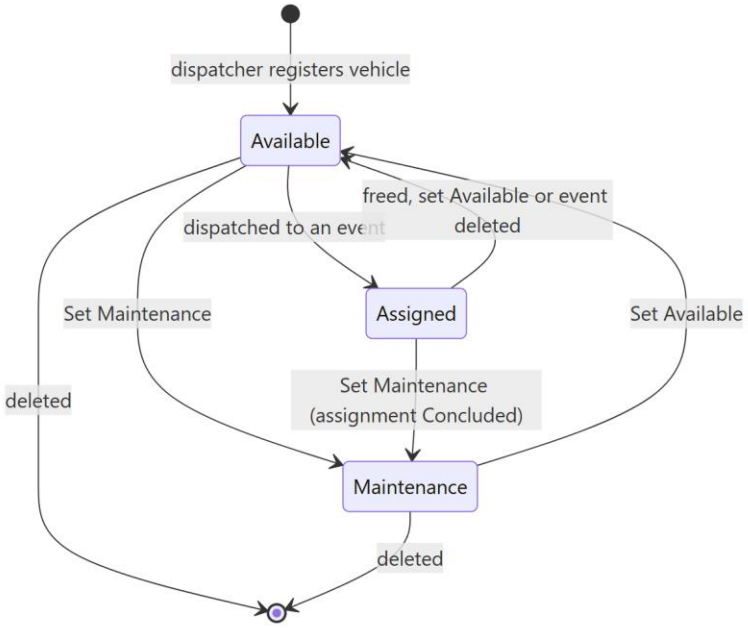



*Figure 13: State diagram of a vehicle*

## 3.2 Description for Component “Administration, Reports and Reviews”

### 3.2.1 Processing narrative (PSPEC) for component

The admin keeps the platform clean. The User Accounts page lists every account with its role, sign-up date, how old the account is and whether it is banned. For staff it also shows the venue and the manager email. The admin can ban and restore accounts, but not their own account and not another admin.

Every other role has a Report button at the bottom of the side menu. The report form changes with the role, so a customer sees choices like 'Safety concern' while a dispatcher sees 'Vehicle misuse or damage'. Reports reach the admin's inbox, oldest first, where the admin can resolve them or ban the offender in one step.

We put reviews in this component too, because they are also about trust. A customer can rate a past experience from 0 to 5 in half steps, and the explore page shows the average for each event.

### 3.2.2 Component interface description

The admin's home screen has User Accounts, Reports and Bookings. The Bookings page lists every booking in the system and lets the admin cancel any of them with a reason. Customers find their past trips under My Experiences.

| Endpoint | Sends | Gets back |
|---|---|---|
| GET /api/admin/users?admin_email= | | all accounts |
| POST /api/admin/users/ban | admin_email, email, banned | success or reason |
| POST /api/reports | reporter, category, offender, summary, details | report id |
| GET /api/admin/reports, POST .../resolve | admin_email, resolution | reports, success |
| POST /api/reviews, GET /api/events/ratings | booking_id, rating, comment | thanks, averages |

*Table 21: Server endpoints used by “Administration, Reports and Reviews”*

### 3.2.3 Component processing detail

- Start
- User presses Report and fills in the form
- System saves the report (at most 5 open per user)
- Admin opens Reports and reads the oldest open report
- Admin resolves it or bans the offender
- System blocks the banned account from logging in and from changing data
- End

### 3.2.3.1 Design Class hierarchy for component Administration, Reports and Reviews

AdminWelcomeScreen builds the three admin pages, and ReportView builds the report form from a table of choices per role (REPORT_OPTIONS). On the server we use is_admin, active_role, get_all_users, set_user_banned, create_report, get_reports, resolve_report and submit_review.

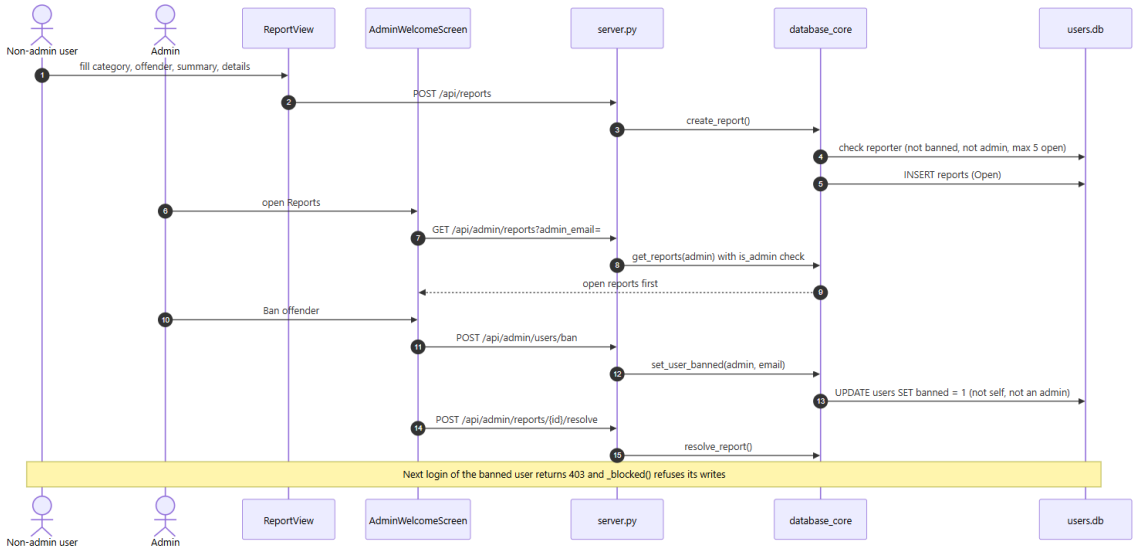



*Figure 14: Sequence diagram: a report leads to a ban*

Figure 14: after the ban the user cannot log in, and every request that changes data is refused by the _blocked guard in server.py.

### 3.2.3.2 Restrictions/limitations for component Administration, Reports and Reviews

Admin requests carry the admin's email. The server checks that this email belongs to an active admin, but without tokens it cannot prove who sent the request. We explain this more in section 5.

### 3.2.3.3 Performance issues for component Administration, Reports and Reviews

The user list and the report list are each read with a single query.

#### 3.2.3.4 Design constraints for component Administration, Reports and Reviews

Admins cannot file reports, and nobody can report their own account. Reviews are only allowed for bookings that were checked in or whose date has passed.

#### 3.2.3.5 Processing detail for each operation of component Administration, Reports and Reviews

The component has the operations listed below:

```
+ set_user_banned(admin_email: str, email: str, banned: bool): dict
- _blocked(email: str, roles): response or None
+ submit_review(email: str, booking_id: int, rating: float, comment: str):
dict
```

##### 3.2.3.5.1 Processing narrative (PSPEC) for each operation

**1. set_user_banned()** is invoked by Ban and Restore in User Accounts and by Ban offender in Reports.

**2. _blocked()** is invoked at the start of every endpoint that changes data.

**3. submit_review()** is invoked from My Experiences.

##### 3.2.3.5.2 Algorithmic model (e.g., PDL) for each operation

**1. set_user_banned()**

```
Start
If the caller is not an active admin, refuse
If the admin is banning themself or another admin, refuse
Save banned, the time and the admin's email (or clear them to restore)
End
```

**2. _blocked()**

```
Start
Find the role of the email, if the account exists and is not banned
If there is no such account, answer 403
If roles are given and the role is not one of them, answer 403
Otherwise let the request continue
End
```

**3. submit_review()**

```
Start
Check the rating is 0 to 5 in steps of 0.5 and the comment is short enough
Check the booking belongs to the user and was attended
Save the review, or update it if one already exists
End
```

## 3.3 Dynamic Behavior for Components

This part shows how the classes work together while the program runs.

### 3.3.1 Interaction Diagrams

We placed each sequence diagram next to the component it belongs to, so the reader does not have to jump back and forth. For a quick overview, these are the interaction diagrams in this document:

- Figure 8: a customer books an event and pays (Booking and Payment).
- Figure 9: a partner pays for the shopping list (Booking and Payment).
- Figure 10: a booker cancels and gets a refund (Cancellations and Refunds).
- Figure 12: staff check a ticket in (Venue Operations).
- Figure 14: a report leads to a ban (Administration, Reports and Reviews).

### 3.3.2 State diagram of a booking

A booking moves through a few states (Figure 15). It starts as Confirmed when it is paid for. From there it is checked in by staff, cancelled under the refund rules, or it simply expires when its date passes. Checked in and Expired are not stored in the database. They are worked out from the ticket and the date each time the booking is read.

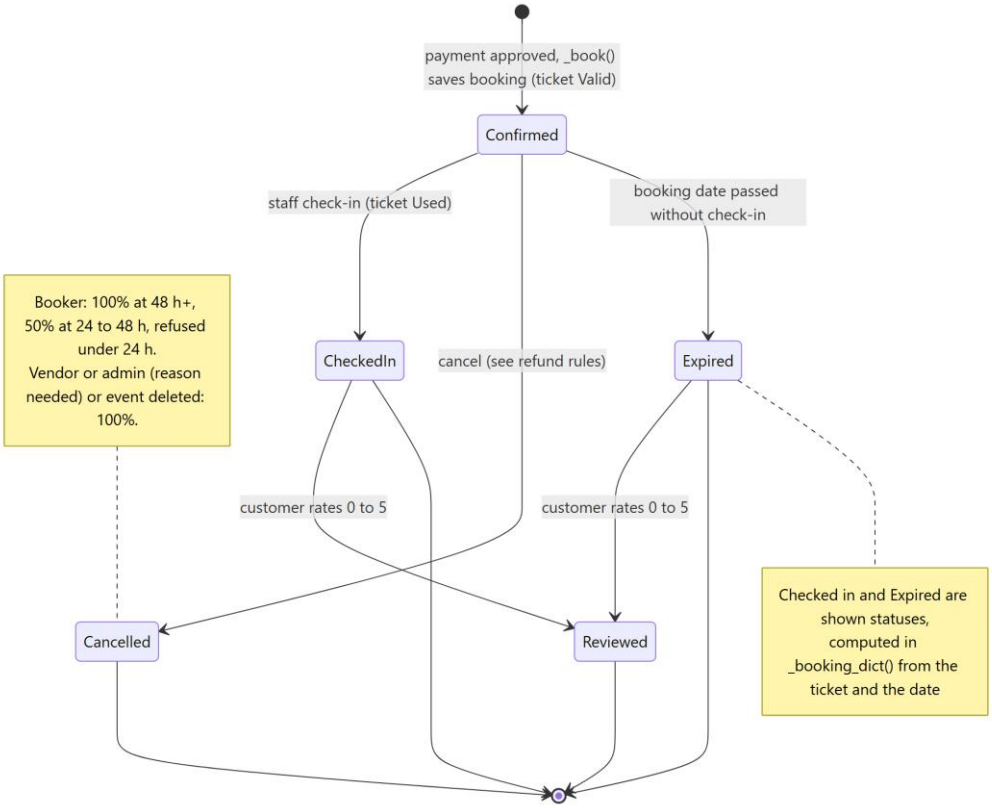



*Figure 15: State diagram of a booking and its ticket*

Section 4

# 4.0 User interface design

A description of the user interface design of Rihla is presented in this section.

## 4.1 Description of the user interface

### 4.1.1 Screen images

Rihla is a desktop app that opens full screen. We wanted it to feel calm and a little bit like a travel brochure, so the login and sign-up screens are split in two: a photo of Dubai with the Rihla logo on the left and a white form on the right. After login, every role gets the same frame: a dark blue side menu with the role's pages, a white top bar with the signed-in email and a live clock, and a light grey area for the content. Blue buttons are for normal actions and orange ones for the main action of a page, like Proceed to Booking or Pay.

Below are the screens of the app with the demo data loaded. The objects on them and what happens when you use them are described in 4.1.2.

#### 4.1.1.1 Signing in and creating an account

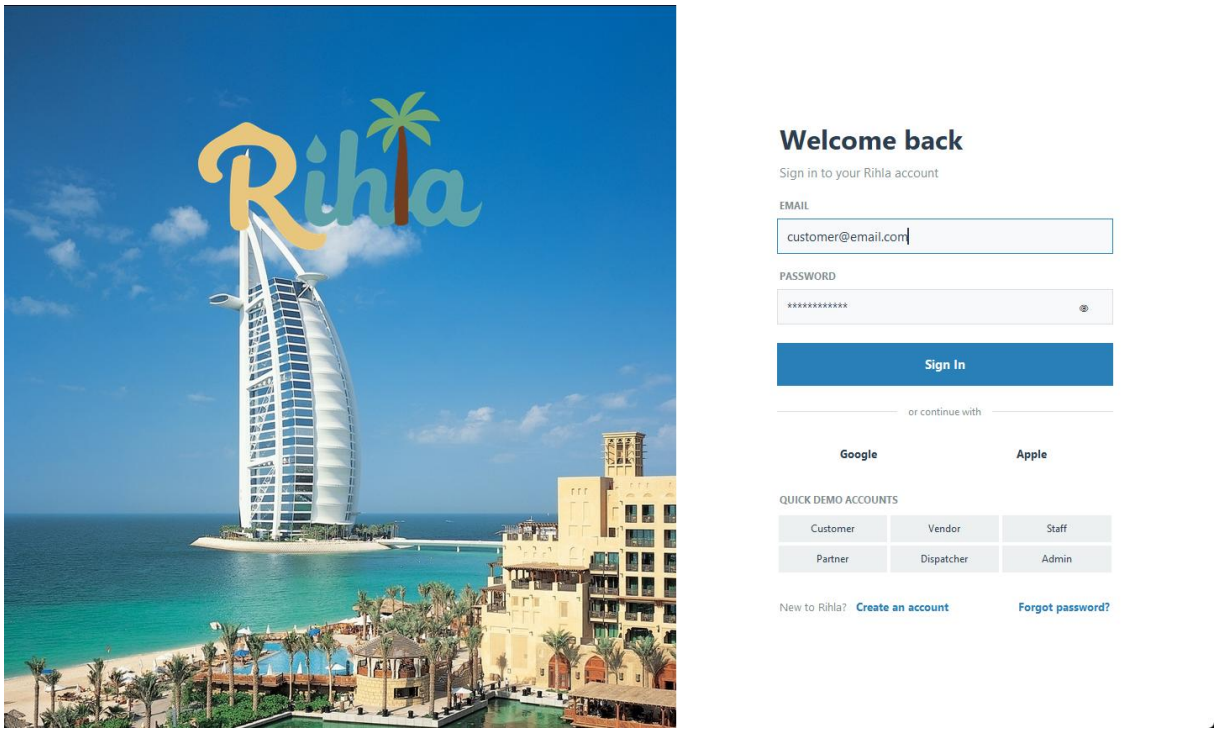



*Figure 16: Login screen with Google and Apple buttons and quick demo accounts*

The quick demo chips under the login form fill in the email and password of each role. They were meant for testing, but we kept them because they make the demo much faster.

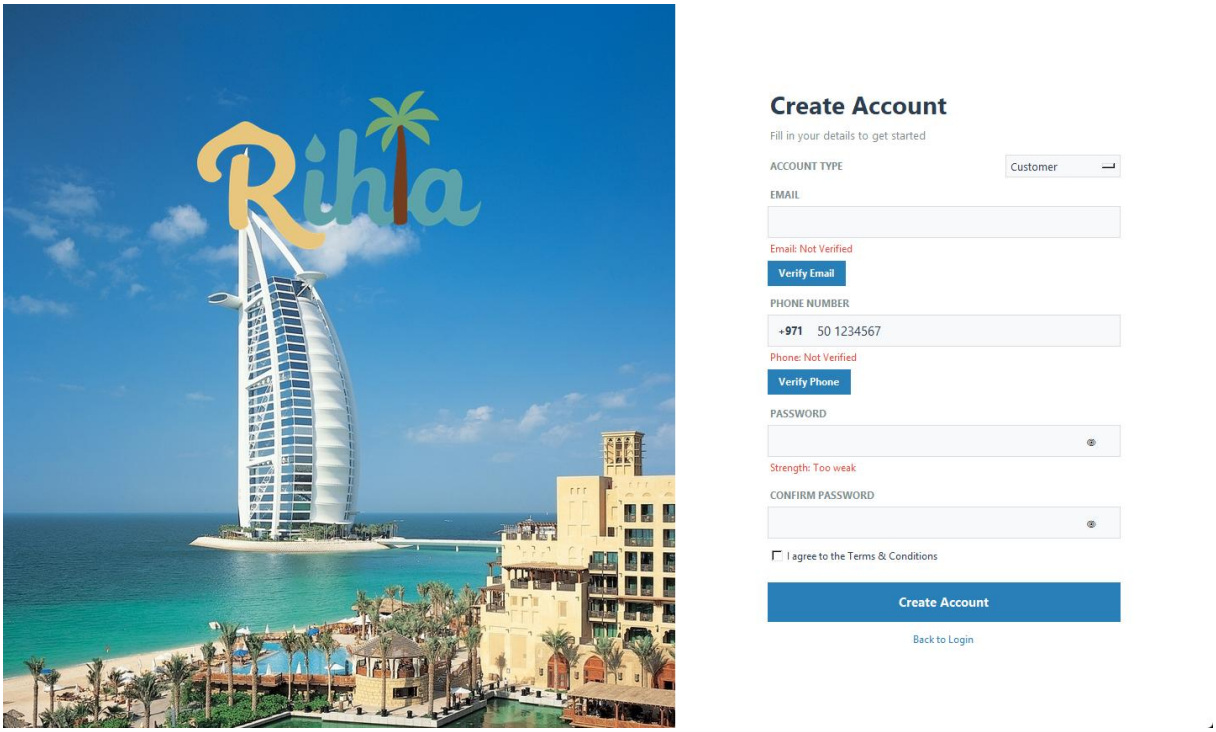



*Figure 17: Create Account with the account type picker*

### 4.1.1.2 Customer pages

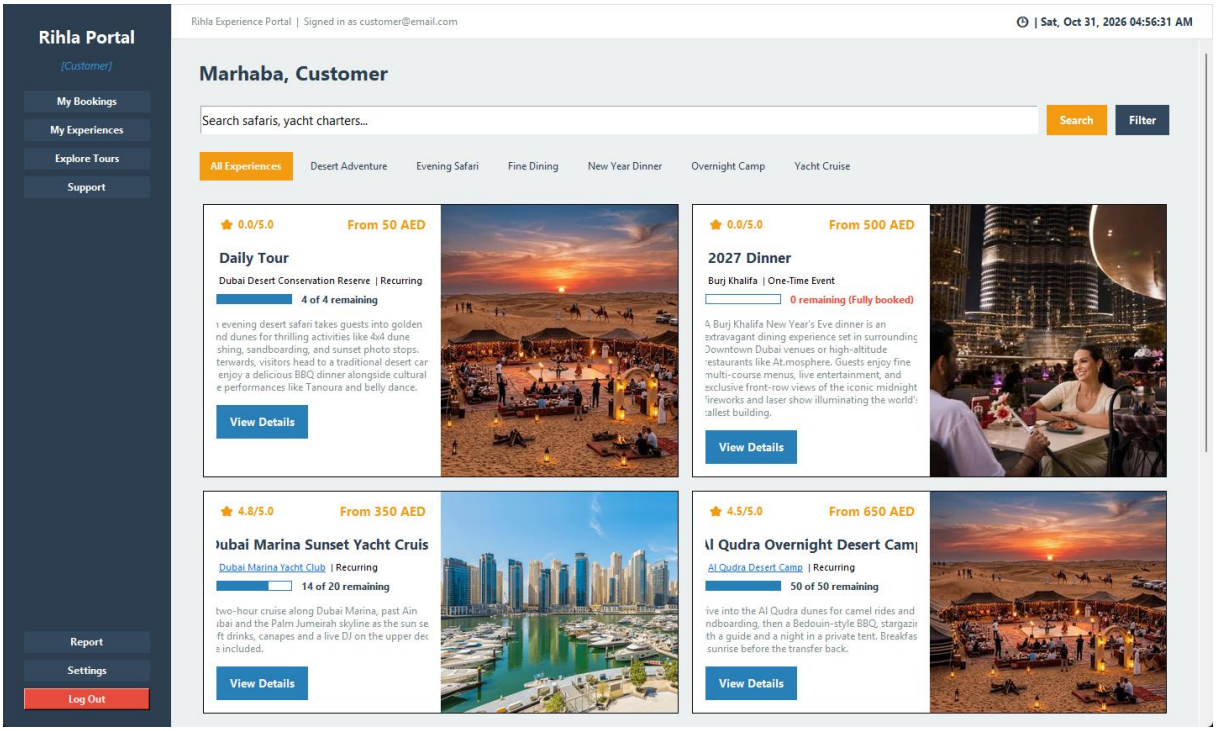



*Figure 18: Explore Tours: categories, search, ratings and capacity bars*

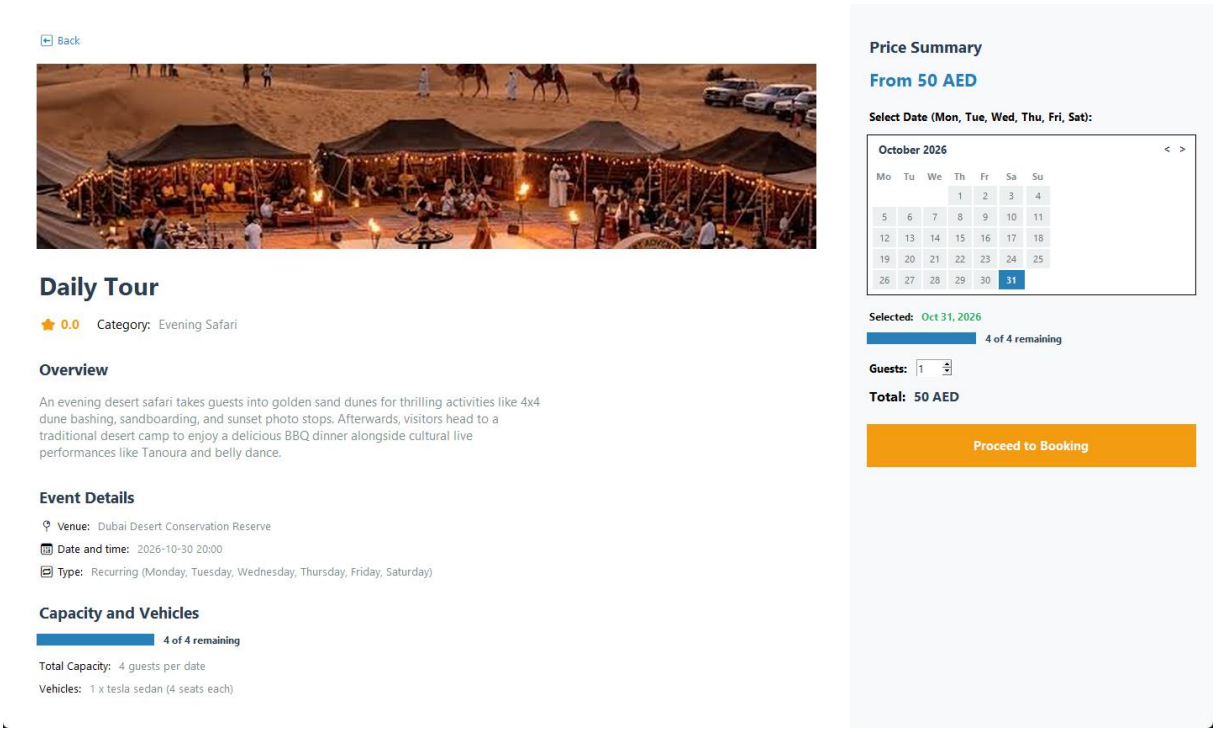



*Figure 19: Event page: picture, details, seats, calendar and price summary*

On the event page only the dates the event really runs on can be clicked. The capacity bar changes when another date is picked.

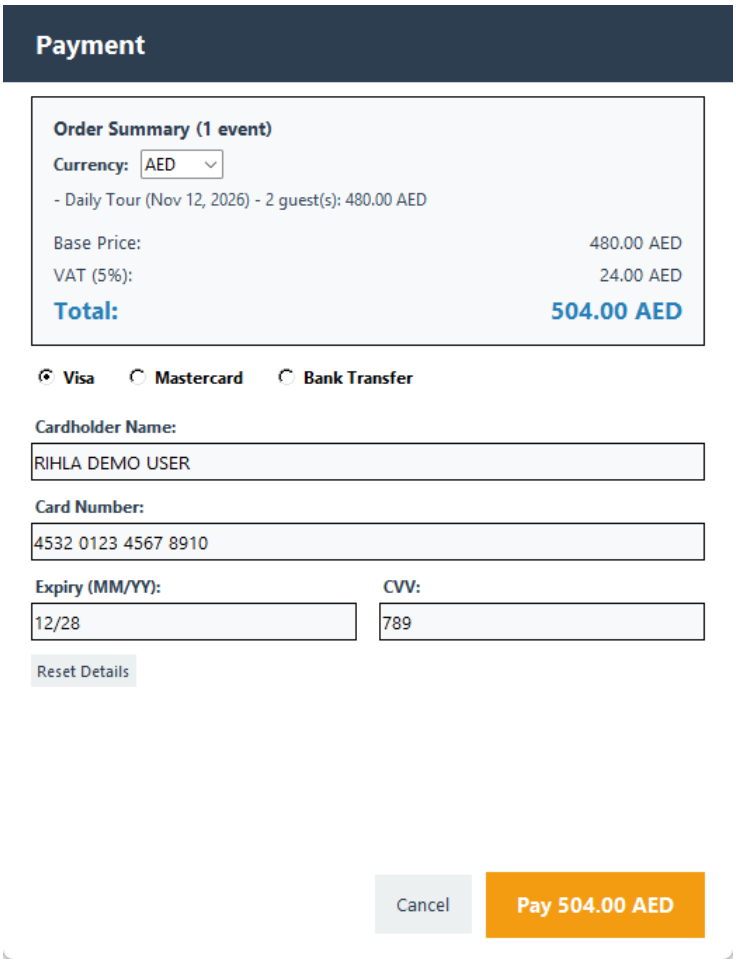



*Figure 20: Payment window with currency, VAT line and prefilled demo card*

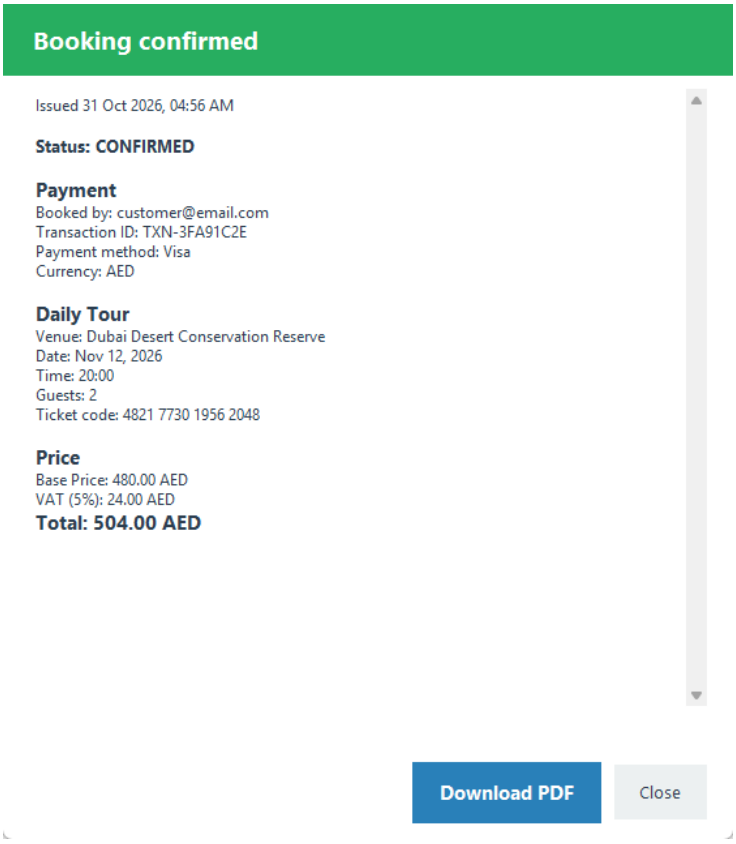



*Figure 21: Booking Details after payment, with Download PDF*

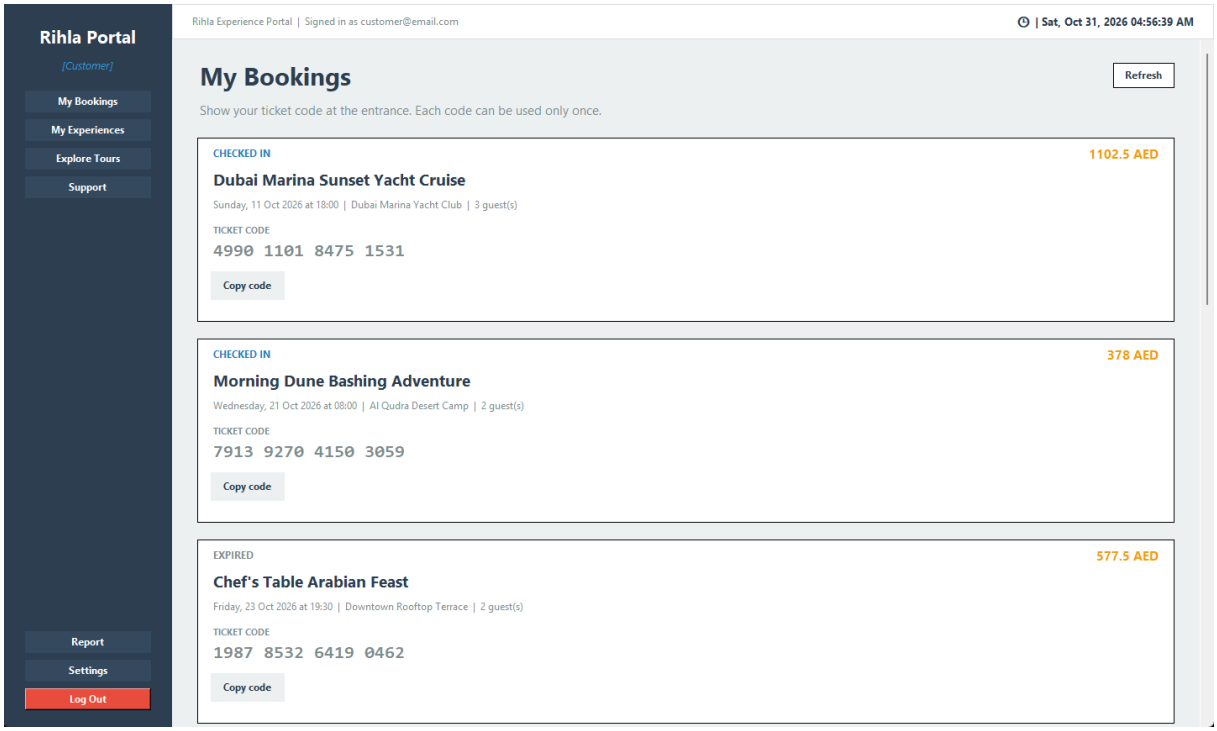



*Figure 22: My Bookings with ticket codes and Copy code*

## 4.1.1.3 Vendor pages

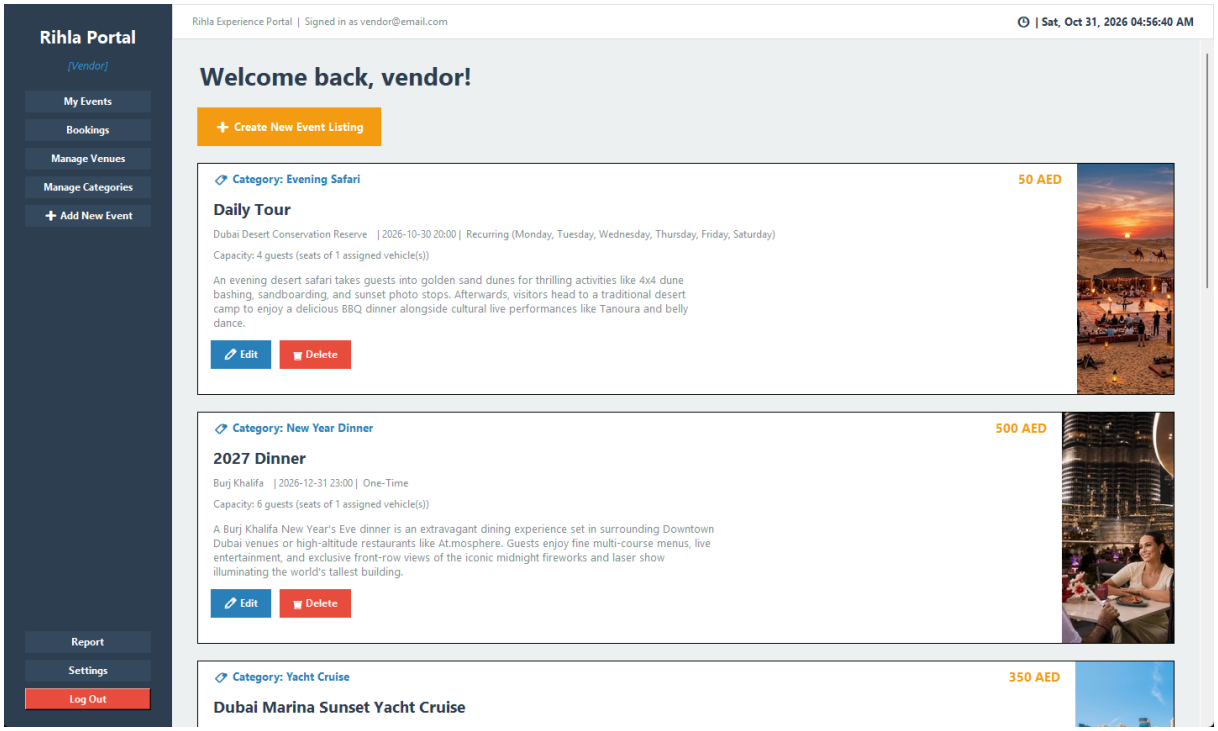



*Figure 23: My Events with pictures, Edit and Delete*

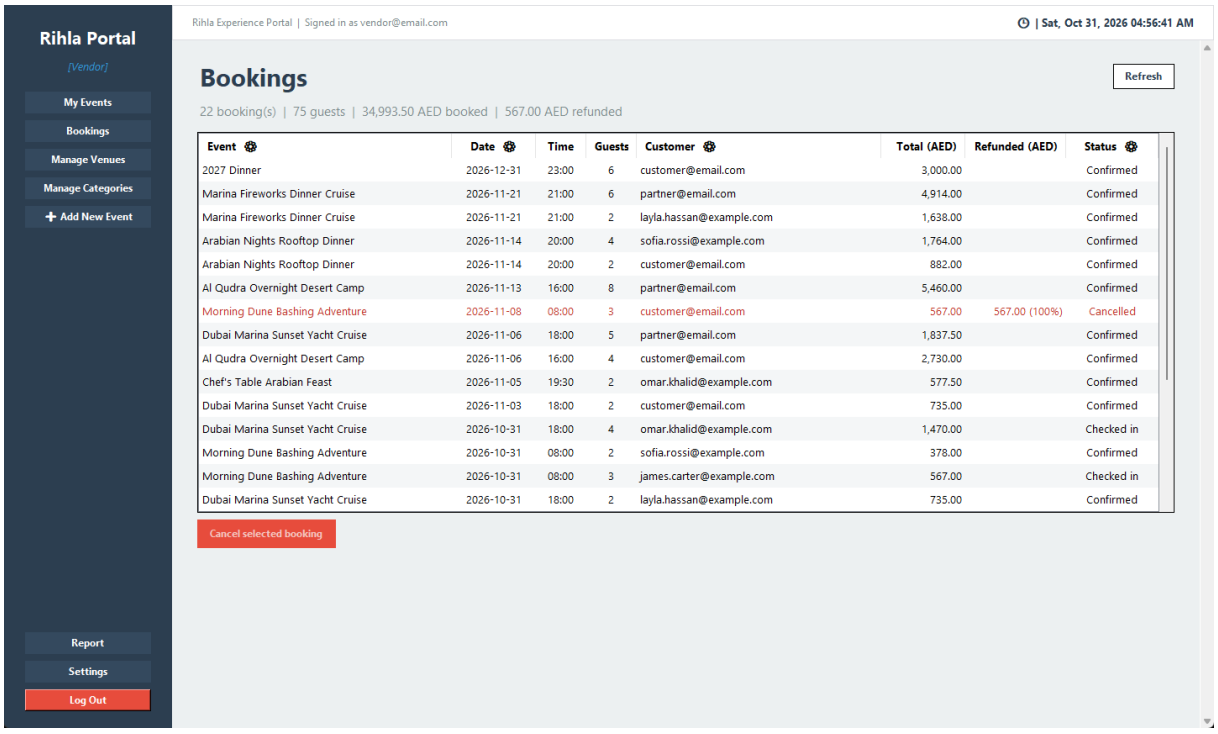



*Figure 24: Vendor Bookings table with column filters and a cancelled booking*

### 4.1.1.4 Staff, partner and dispatcher pages

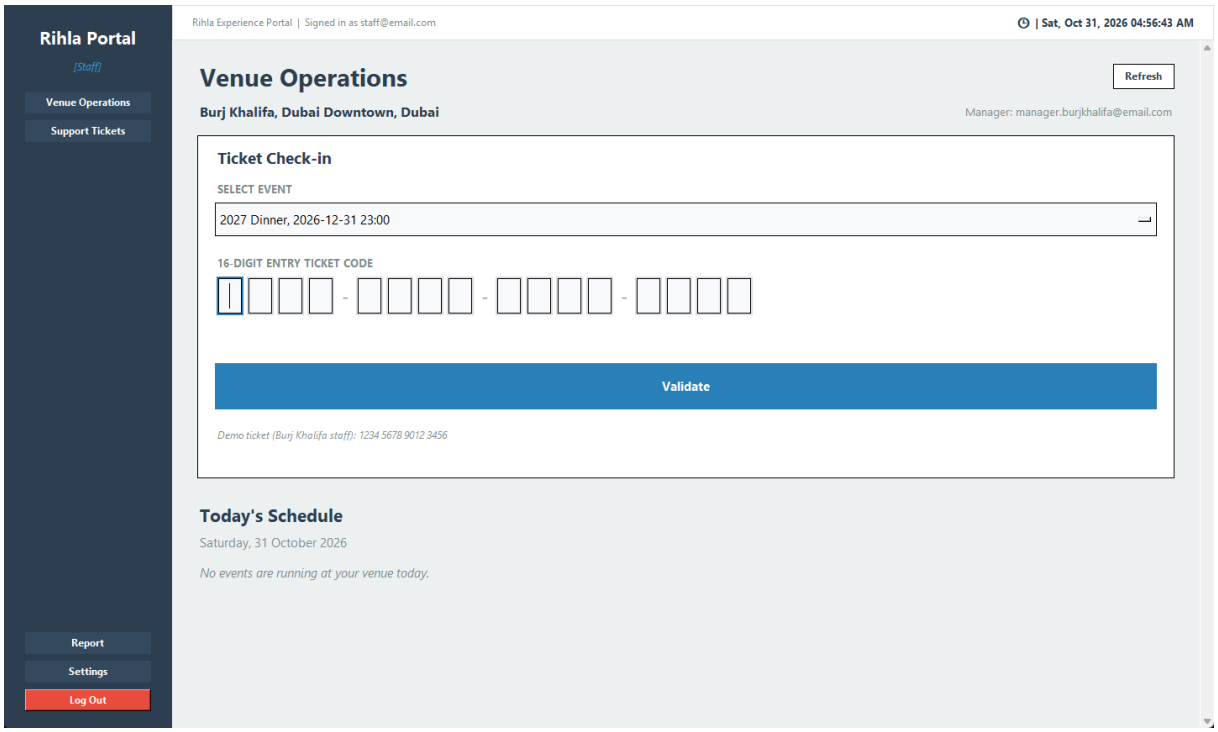



*Figure 25: Venue Operations: venue, check-in and today's schedule*

In this screenshot the schedule is empty because no event at Burj Khalifa runs on the day it was taken.

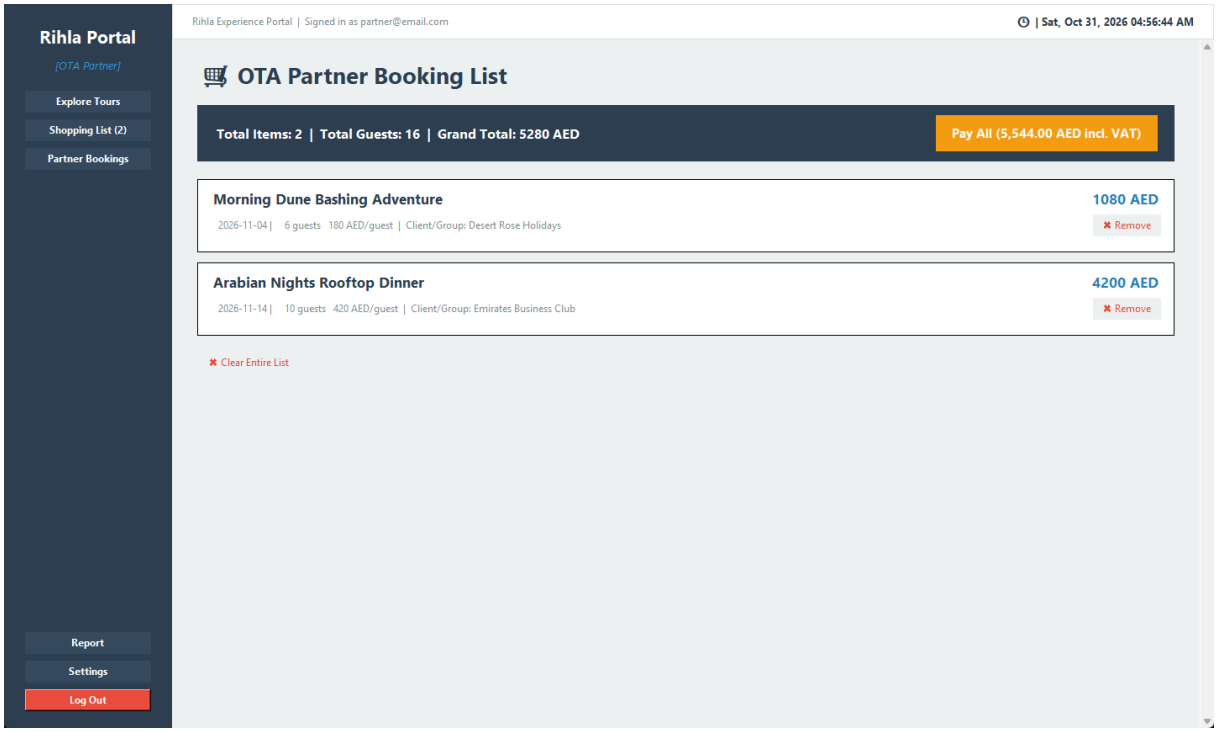



*Figure 26: Partner shopping list paid in one go*

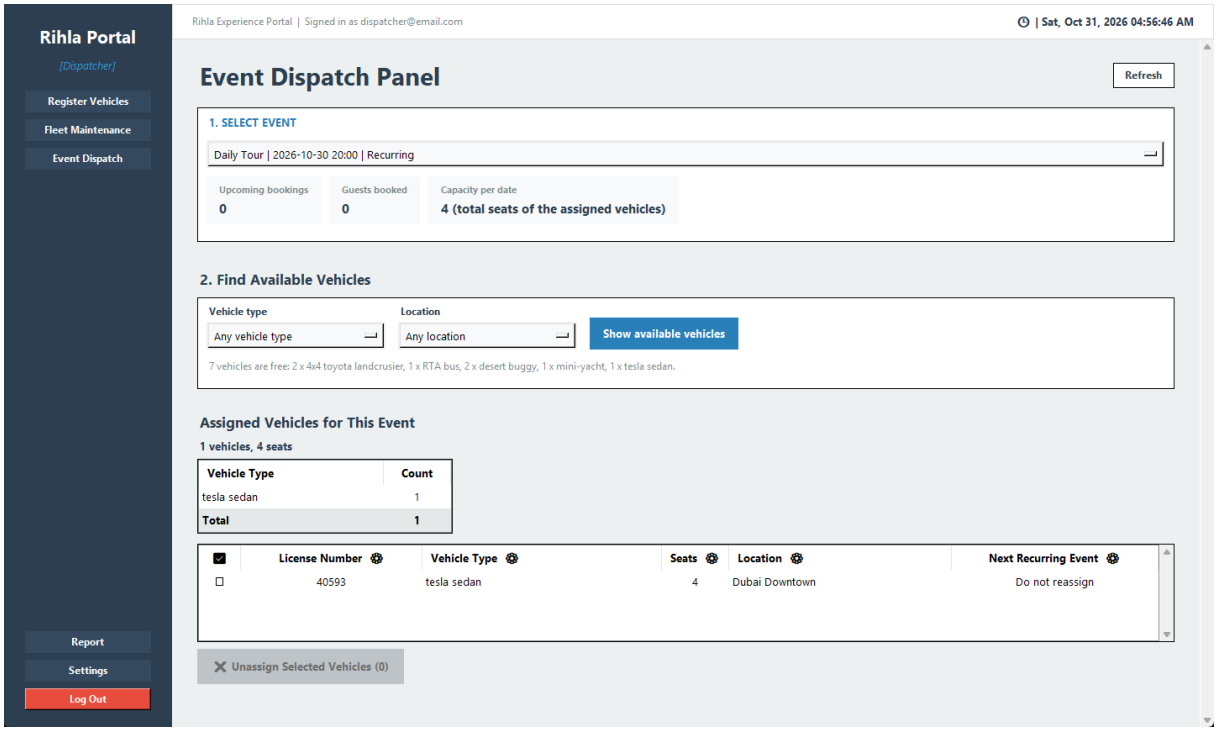



*Figure 27: Event Dispatch with available and assigned vehicles*

## 4.1.1.5 Admin pages and reports

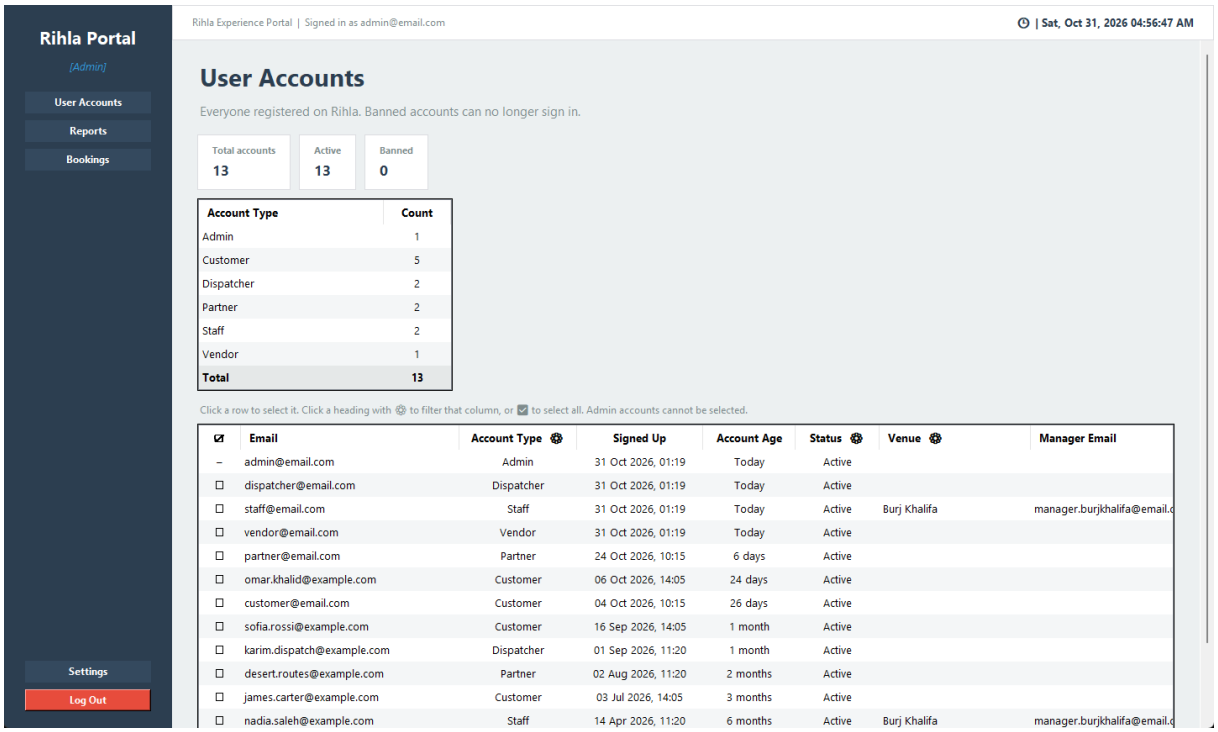



*Figure 28: User Accounts with counts per role, sign-up date, age and status*

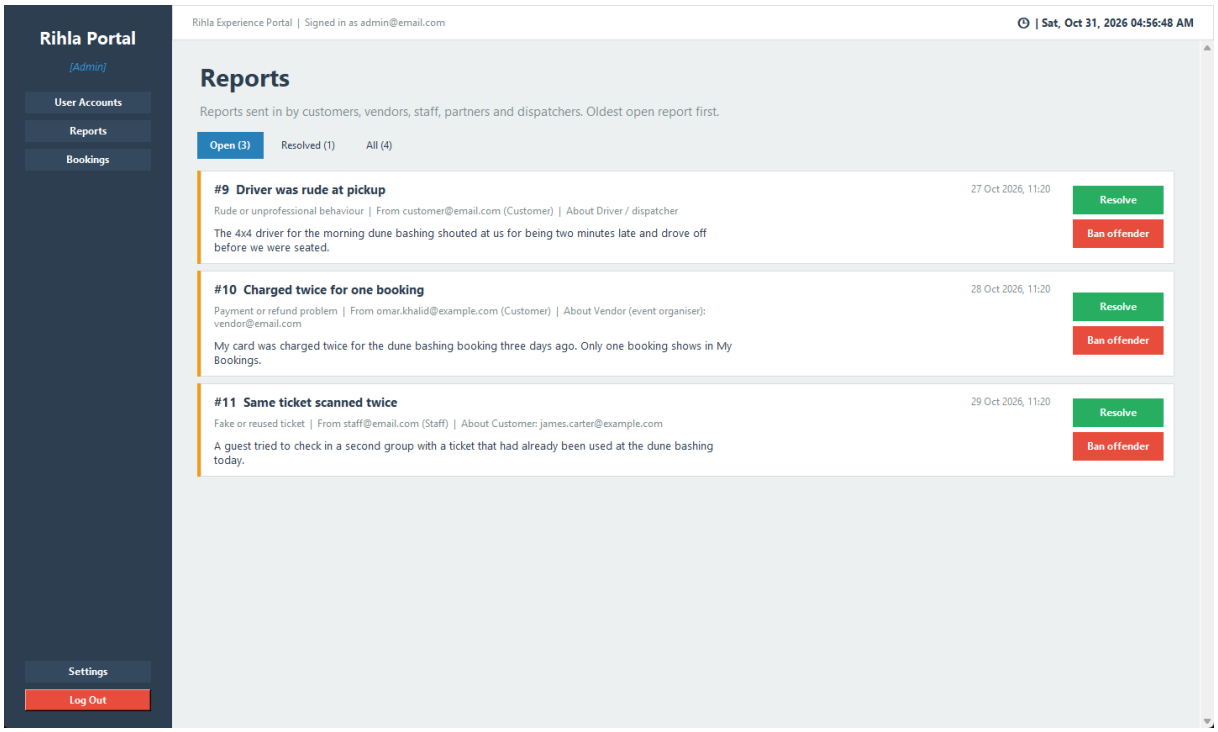



*Figure 29: Reports inbox with Resolve and Ban offender*

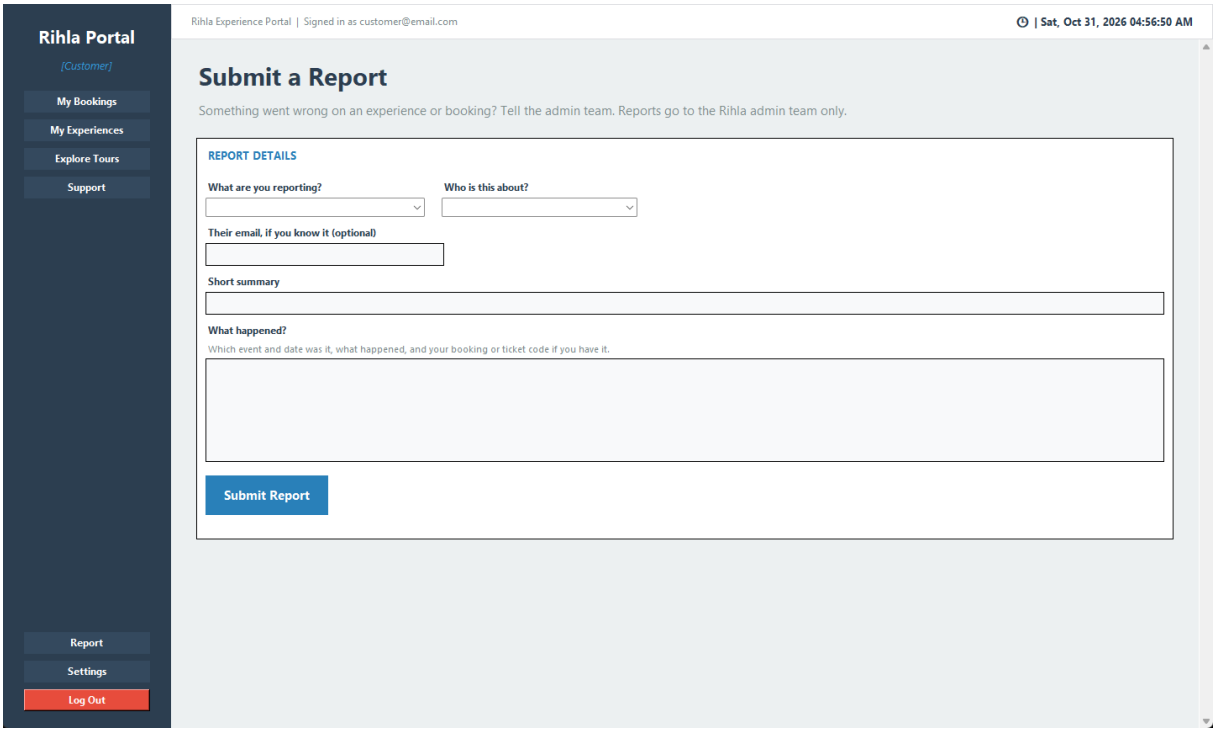



*Figure 30: The Report form that every non-admin role can open*

### 4.1.2 Objects and actions

The interface of Rihla is built from a small set of objects that come back on most screens:

- **Side menu:** the role's pages at the top, Report, Settings and Log Out at the bottom. Clicking a page replaces the content area.
- **Top bar:** shows who is signed in and the date and time, updated every second.
- **Event cards:** picture, title, venue, rating, price and a capacity bar. View Details opens the event page.
- **Category chips and search box:** filter the list of events. The Filter window adds a maximum price slider.
- **Calendar:** only allowed dates can be clicked, and the chosen date updates the seats and price.
- **Capacity bar:** a bar with 'N of TOTAL remaining', orange 'Hurry up!' when few seats are left and red 'Fully booked' at zero.
- **Digit boxes:** one box per digit for the SSN and ticket codes. They jump to the next box and accept a pasted code.

- **Payment window:** currency box, base price, VAT and total, payment method buttons and Pay.
- **Tables:** used by vendors, dispatchers and the admin. Columns with a gear icon can be filtered by clicking the heading, and the first column has check boxes.
- **Booking cards:** status, ticket code in groups of four digits, Copy code, the refund policy line and Cancel.
- **Pop-up windows:** for confirmations, the cancel reason, reviews, settings and errors.
- **Refresh buttons:** on the pages that show changes made by other people, such as bookings and dispatch.

## 4.2 Interface design rules

We followed the eight golden rules from the lectures. Below is how each one shows up in Rihla.

1. **Strive for consistency.** All roles share one frame (BaseWelcomeScreen), one colour file (theme.py) and one table style (table_style.py). A table looks the same for a vendor and for the admin.
2. **Seek universal usability.** New users can try the app with the demo chips. Prices can be shown in four currencies, which matters for tourists.
3. **Offer informative feedback.** The capacity bar, the password strength label and messages such as 'Only 3 of 20 seats are left on that date' tell the user what is going on. A declined payment is explained inside the payment window instead of a separate pop-up.
4. **Design dialogs to yield closure.** Every booking ends with the Booking Details window, which sums up the whole booking and offers the PDF.
5. **Prevent errors.** Wrong dates cannot be picked, digit boxes only take digits, and the sign-up form hides the SSN and venue fields when they do not apply. The server still checks everything again.
6. **Permit easy reversal of actions.** Bookings can be cancelled under clear rules, cart items can be removed, banned users can be restored and the payment form has Reset Details.

7. **Keep users in control.** The refund is shown before the user confirms a cancellation, and pages with shared data have a Refresh button.
8. **Reduce short-term memory load.** Ticket codes are split into four groups of four digits and have a Copy code button. The signed-in email is always visible in the top bar.

## 4.3 Components available

We did not use a design tool for the screens. Everything is built with the standard tkinter library and ttk (Frame, Label, Button, Entry, Text, Canvas, Scrollbar, Scale, Spinbox, Radiobutton, Checkbutton, OptionMenu, Toplevel, Combobox and Treeview), plus Pillow for pictures and the optional tksvg for the Lucide icons. Where tkinter had nothing suitable we wrote small widgets ourselves: DigitBoxes, CapacityBar and the picture cards in event_images.py.

## 4.4 UIDS description

We did not have a user interface development system. The screens are written by hand in Python. To keep them consistent we rely on shared code instead: theme.py for colours and fonts, table_style.py for tables, navigate.py and routing.py for moving between screens. Adding a new role only needs a new welcome module and one line in routing.py.

Section 5

# 5.0 Restrictions, limitations, and constraints

This section lists the things that limited our design and the weak points we know about. We prefer to name them here rather than have them found during the demo.

**Design constraints and restrictions:**

- All rules about seats, money and access must live on the server, not in the screens.
- All money is kept in AED and the 5% VAT is always shown on its own line.
- Sensitive data such as passwords and SSNs is only stored as salted hashes.
- The app must run on a normal Windows laptop with Python 3.12 and no database server to install.
- The demo must be repeatable, so the seed data is relative to today and can be cleared again.

**Major factors that hindered the design:**

1. **No login tokens.** The server does not hand out a session token after login. Requests carry an email and the server trusts it. It does check that the account exists, is not banned and has the right role, but someone who writes their own HTTP requests could act as another user. This is the biggest gap we would close next.
2. **Missing ownership checks.** The password change endpoint does not ask for the old password. Updating and deleting events, deleting venues, and the vehicle and dispatch endpoints do not check that the record belongs to the caller.
3. **Local only.** The server runs on 127.0.0.1 with plain HTTP and Flask's development server. It is made for one machine and is not ready for the internet.
4. **Demo payments.** Payments only compare the form with fixed details, and refunds are only recorded. A real payment provider would also need the booking to be reserved before the money is taken.

5. **Fixed currency rates.** USD, EUR and GBP use fixed numbers in pricing.py, so the shown prices can be out of date.
6. **Fake verification.** Email, phone and password reset codes are always 000000, and the Google and Apple buttons are ordinary email logins.
7. **SQLite.** One file is easy to set up, but only one writer at a time is allowed. Our transactions keep the data correct, but many users at once would have to wait.
8. **Email as a key.** Many tables point to users by email, so an email address cannot be changed.
9. **Prices as text.** Prices like '240 AED' are stored as text and only the digits are read, so prices with decimals are not possible.
10. **Server time.** All times are the local time of the server machine.
11. **Phase 1 features we did not build.** QR codes, the offline driver app, live vehicle tracking, email and SMS messages and Arabic support.

Section 6

# 6.0 Testing Issues

Unlike Phase 1, we now have working code, so we used both white-box and black-box testing. The automated tests are in the tests folder and use Python's unittest. They point the database to a temporary file, so the real data is never touched. Most tests call the server functions directly, and some go through the Flask API with app.test_client() to check the role guards. There are 70 tests, and on 31 October 2026 all 70 passed. They run with:

```
python -m unittest discover -s tests
```

Next to the automated tests, we went through the demo script in README.md role by role after filling the database with dummy_data.ps1.

## 6.1 Classes of tests

**1. Unit testing (white box)**

**Expectations:** Each rule should work on its own, including at its limits.

**Example:** refund_percent_for gives 100 at exactly 48 hours, 50 between 48 and 24, and nothing below 24 (CancellationTests). Two vehicles of the same type add their seats together (CapacityTests).

**2. Functional testing (black box)**

**Expectations:** Every function should do what the user expects, judged only by the result.

**Example:** Booking 3 guests for a 240 AED event stores 756 AED and a 16-digit code (BookingTests).

**3. Transaction testing**

**Expectations:** A group of actions should either all happen or not happen at all.

**Example:** A partner cart with one full event books nothing and keeps the cart (test_failed_cart_checkout_keeps_no_bookings).

**4. Security testing**

**Expectations:** Only the right, active accounts should be able to act.

**Example:** A banned or unknown account cannot book or open support tickets, and only partners can use the cart (ApiAccountGuardTests). Staff cannot check in another venue's ticket (StaffVenueTests).

**5. Upgrade testing**

**Expectations:** Databases made by older versions of our code should still work.

**Example:** Tests build an old database without the new columns and check that init_db adds them (for example capacity, ban columns, staff venues).

**6. Interface testing**

**Expectations:** Small widgets should show the right numbers.

**Example:** The capacity bar shows 'N of TOTAL remaining' with the right ratio, and the demo payment fails as soon as a field is changed (PartnerAndPaymentTests).

## 6.2 Expected software response

**Expected response through equivalence partitioning:**

| Restriction | User | Valid classes | Invalid classes |
|---|---|---|---|
| Customer books 1 to 10 guests | Customer | 1, 5, 10 | 0, 11 |
| Partner has no 10 guest limit | Partner | 11, 25 (if seats are left) | more than the seats left |
| Booking date is today or later and fits the event | Customer, Partner | Tuesday for a Tue/Thu event | yesterday, a Monday |
| SSN is 16 digits, not one digit repeated | Vendor, Staff, Partner, Dispatcher | 4000100020003001 | 15 digits, 1111111111111111, letters |
| Password has at least 8 characters | All | customer1234 | abc123 |
| Rating is 0 to 5 in steps of 0.5 | Customer | 0, 3.5, 5 | 4.3, 5.5, -1 |
| Event capacity is 1 to 5000 | Vendor | 1, 30, 5000 | 0, 5001 |
| Cancel reason up to 300 characters, required for operators | Vendor, Admin | a short reason | empty, 301 characters |

*Table 22: Equivalence classes*

**Expected response for the main test cases:**

| Test case | Input | Expected response |
|---|---|---|
| Booking price | 3 guests, 240 AED event | Base 720, VAT 36, total 756 AED and a 16-digit code |
| Full event | More guests than seats left | Refused with the seats left and the total |
| Vehicle seats | Two 4x4 Land Cruisers assigned | Capacity becomes 12 |
| Check-in twice | Same valid code twice | First: access granted. Second: already used |
| Other venue | Al Qudra staff, Burj Khalifa ticket | Refused: not for an event at your venue |
| Cancel early | Customer, 3 days before | 100% refund, seats free again |
| Cancel late | Customer, 30 hours before | 50% refund, VAT refunded in the same share |
| Cancel too late | Customer, 10 hours before | Refused: closes 24 hours before |
| Operator without reason | Vendor cancels with no reason | Refused: a reason is required |
| Edited card | One digit of the card changed | Payment declined, shown in the window |
| Admin sign-up | Role Admin at sign-up | Never creates an admin account |
| Banned user | Login and booking after a ban | Both answered with 403 |
| Self ban | Admin bans their own email | Refused |

*Table 23: Expected responses*

## 6.3 Performance bounds

- **Response time:** a booking, cancellation or check-in is one short SQLite transaction and shows its result without a visible delay on our laptops.
- **Timeouts:** the client waits at most 10 seconds for the server and then tells the user it cannot connect, instead of freezing.
- **Pictures:** at most 1400 pixels after upload and 4 MB on the server, so the event lists stay quick.
- **Limits:** 10 guests per customer booking, 5000 seats per event, 3 open support tickets per customer, 5 open reports per user, 300 characters per cancel reason and 500 per review comment.
- **Test run:** the full suite takes about five minutes, mostly because some tests open windows and the password hashing is slow on purpose.

## 6.4 Identification of critical components

These are the parts we test most carefully, because a mistake there costs someone money, a seat or their trust in the app:

1. **Seat check (_book and _capacity_info).** If it fails, two people can get the same seat, which is the exact problem Rihla was made to solve.

2. **Refund rules (_cancel, refund_percent_for, split_refund).** If they fail, customers get the wrong amount back. The 48 and 24 hour edges must be exact.
3. **Cart checkout.** If it fails, a partner could pay for a group and only get part of the bookings.
4. **Ticket check-in.** If it fails, one ticket could let people in twice, or at the wrong venue.
5. **Access guards (login_status, _blocked, set_user_banned).** If they fail, banned people keep using the app.
6. **SSN handling.** If it fails, a real SSN could end up stored as plain text.
7. **Database upgrade (init_db).** If it fails, the server would not start on an older database.

# 7.0 Work distribution

| Team member | Part of this document |
|---|---|
| Salem Al Tamimi | Section 1 Introduction and Section 2 Data design |
| Hamzah and Hamad | Section 3 Architectural and component-level design |
| Mir Umer | Section 4 User interface design |
| Omar Arif | Section 5 Restrictions, limitations, and constraints |
| Oumar Ibrahim | Section 6 Testing issues |

*Table 24: Who wrote which part*